

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Южно-Уральский государственный университет
(национальный исследовательский университет)»**

Высшая школа электроники и компьютерных наук

Кафедра системного программирования

РАБОТА ПРОВЕРЕНА

Рецензент

Руководитель Бюро аналитических
приложений ООО «Компас Плюс»

_____ В.В. Котрачев

«___» _____ 2025 г.

ДОПУСТИТЬ К ЗАЩИТЕ

Заведующий кафедрой, д.ф.-м.н.,
профессор

_____ Л.Б. Соколинский

«___» _____ 2025 г.

**Разработка веб-приложения «Конструктор правил
для автоматического выявления подозрительных транзакций
в системе противодействия банковским мошенничествам»**

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ЮУрГУ – 09.04.04.2025.308-554.ВКР**

Научный руководитель,
зав. каф. СП, д.ф.-м.н., профессор
_____ Л.Б. Соколинский

Автор работы,
студент группы КЭ-228
_____ М.А. Дремин

Ученый секретарь
(нормоконтролер)
_____ И.Д. Володченко
«___» _____ 2025 г.

Челябинск, 2025 г.

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
**«Южно-Уральский государственный университет
(национальный исследовательский университет)»**
Высшая школа электроники и компьютерных наук
Кафедра системного программирования

УТВЕРЖДАЮ

Зав. кафедрой СП

_____ Л.Б. Соколинский

27.01.2025 г.

ЗАДАНИЕ

на выполнение выпускной квалификационной работы магистранта

студенту группы КЭ-228

Дремину Михаилу Александровичу,

обучающемуся по направлению

09.04.04 «Программная инженерия»

1. Тема работы (утверждена приказом ректора от 21.04.2025 г. № 648-13/12)

Разработка веб-приложения «Конструктор правил для автоматического выявления подозрительных транзакций в системе противодействия банковским мошенничествам».

2. Срок сдачи студентом законченной работы: 02.06.2025 г.

3. Исходные данные к работе

3.1. Кряжева Е. В., Васина Т. А. Общие подходы к проектированию веб-приложений. // Заметки ученого, 2021. № 9–2. С. 32–36.

3.2. ASP.NET Core Blazor | Microsoft. [Электронный ресурс] URL: <https://learn.microsoft.com/en-us/aspnet/core/blazor/?view=aspnetcore-7.0> (дата обращения: 27.01.2025 г.).

3.3. Himschoot P. Microsoft Blazor. // Berkeley, CA: Apress, 2022. 90 p.

4. Перечень подлежащих разработке вопросов

4.1. Провести анализ предметной области.

4.2. Спроектировать веб-приложение.

4.3. Реализовать веб-приложение.

4.4. Провести тестирование веб-приложения.

5. **Дата выдачи задания:** 27.01.2025 г.

Научный руководитель,
зав. каф. СП, д.ф.-м.н., профессор

Л.Б. Соколинский

Задание принял к исполнению

М.А. Дремин

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	5
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	7
1.1. Фрод-мониторинг.....	7
1.2. Обоснование выбора средств реализации	11
1.3. Сравнительный обзор аналогов.....	11
2. ПРОЕКТИРОВАНИЕ	15
2.1. Функциональные и нефункциональные требования.....	15
2.2. Диаграмма вариантов использования	17
2.3. Проектирование архитектуры системы.....	18
2.4. Диаграмма классов рискового правила	20
2.5. Диаграмма классов версии рискового правила	23
3. РЕАЛИЗАЦИЯ	28
3.1. Программные средства реализации	28
3.2. Реализация серверной части	28
3.2. Реализация клиентской части	29
4. ТЕСТИРОВАНИЕ	49
4.1. Функциональное тестирование	49
4.2. Тестирование системы на адаптивность и кросс-браузерность..	53
ЗАКЛЮЧЕНИЕ	54
ЛИТЕРАТУРА.....	55

ВВЕДЕНИЕ

Актуальность

С ростом количества безналичных операций и развитием цифровых технологий банковские учреждения сталкиваются с увеличением числа мошеннических схем. Для защиты клиентов и предотвращения финансовых потерь банки внедряют системы противодействия мошенничеству, которые позволяют оперативно реагировать на подозрительные транзакции.

Банковские мошенничества представляют собой серьезную проблему для финансовых организаций, поскольку злоумышленники используют различные схемы для обхода стандартных механизмов безопасности. В современных условиях важно не только выявлять подозрительные транзакции, но и оперативно настраивать алгоритмы их обработки, чтобы своевременно предотвращать мошеннические действия.

Одним из ключевых инструментов таких систем являются правила обработки рискованных операций. Однако ручное создание и обновление этих правил требует значительных временных и людских ресурсов, что снижает эффективность борьбы с мошенничеством. Разработка веб-приложения, позволяющего автоматически формировать и управлять такими правилами, позволит повысить оперативность их настройки, снизить зависимость от человеческого фактора и адаптировать систему к новым угрозам.

В данной работе рассматривается разработка веб-приложения, которое позволит специалистам по безопасности создавать и управлять правилами обработки рискованных транзакций. Эти правила будут определять алгоритмы реагирования системы на операции, соответствующие заданным критериям риска.

В связи с этим для компании ООО «Компас Плюс» стала актуальна задача в разработке веб-приложения для создания правил автоматического выявления подозрительных транзакций в системе противодействия банковским мошенничествам.

Постановка задачи

Целью выпускной квалификационной работы является разработка веб-приложения «Конструктор правил для автоматического выявления подозрительных транзакций в системе противодействия банковским мошенничествам».

Для достижения поставленной цели необходимо решить следующие задачи.

1. Выполнить анализ предметной области.
2. Сформулировать требования к приложению.
3. Спроектировать веб-приложение.
4. Реализовать веб-приложение.
5. Провести тестирование веб-приложения.
6. Внедрить веб-приложение в компанию ООО «Компас Плюс».

Структура и содержание работы

Работа состоит из введения, четырех глав, заключения и списка литературы. Объем работы составляет 56 страниц, объем списка литературы – 17 источников.

В первой главе описывается предметная область проекта, обоснование выбора средств реализации и аналогичные системы к приложению.

Во второй главе приведены функциональные и нефункциональные требования к системе, построена диаграмма вариантов использования UML. Также описана архитектура системы и построены необходимые диаграммы UML.

В третьей главе описаны программные средства, которые использовались при разработке веб-приложения. Также описаны детали реализации основного функционала.

В четвертой главе представлены результаты тестирования работы веб-приложения. По результатам тестирования был сделан вывод о достижении поставленных целей.

В заключении представлены результаты выполненной работы.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1. Фрод-мониторинг

Банковские мошенничества представляют собой одну из наиболее актуальных проблем в сфере финансовых технологий. Злоумышленники используют различные схемы для совершения незаконных операций, включая фрод (мошенничества в области информационных технологий) с платежными картами, отмывание денег и другие виды финансовых махинаций. В связи с этим банки и финансовые организации внедряют системы противодействия мошенничеству, целью которых является своевременное обнаружение подозрительных операций и предотвращение ущерба.

Для борьбы с мошенничеством организации используют различные системы и технологии включая [13]:

- 1) мониторинг транзакций – анализ операций в реальном времени;
- 2) оценку рисков – анализ поведенческих и финансовых данных;
- 3) аналитику данных – выявление подозрительных паттернов;
- 4) автоматизированное выявление мошеннических схем с помощью искусственного интеллекта и машинного обучения.

Фрод-мониторинг (антифрод) – это набор технологий и методов, направленных на предотвращение мошенничества в финансовых и интернет-транзакциях. Система анализирует операции с использованием различных критериев и алгоритмов для выявления подозрительных действий. В случае обнаружения отклонений от нормального поведения система инициирует дополнительные проверки и уведомляет о возможных рисках, чтобы минимизировать ущерб от мошенничества [6].

Антифрод-системы бывают двух типов: транзакционные и сессионные. Транзакционные системы проверяют конкретные операции, такие как оплаты или переводы, с целью блокировки мошеннических действий до их завершения. Сессионные системы фокусируются на поведении пользователя, анализируя его действия (например, движение курсора или скорость

набора текста) и выявляя атипичные поведения для предотвращения мошенничества. Существуют также смешанные системы, которые сочетают элементы обоих типов [1].

При выборе антифрод-системы важно учитывать способ ее поставки: облачное решение предоставляет быструю подключаемость и шифрование данных, а локальное размещение предоставляет больший контроль над данными, но требует наличия инфраструктуры и специалистов для поддержки работоспособности программного продукта. Необходимо также учитывать использование истории данных для проверки транзакций и нагрузку на базу данных, а также возможность интеграции с внешними источниками данных, например, черными списками. Важным фактором является частота обновлений программы, чтобы она соответствовала актуальным угрозам мошенничества [1].

Для создания критериев и алгоритмов для выявления подозрительных транзакций используются статические рискованные правила. Статические рискованные правила – это предопределенные условия, используемые для выявления потенциально мошеннических или аномальных действий в системе. Эти правила основываются на фиксированных логических выражениях, определяющих, какие операции или пользователи считаются подозрительными.

Когда несколько статических рискованных правил работают в связке, их цель – комплексная оценка транзакций или поведения пользователей для выявления мошенничества. Каждое из таких правил оценивает отдельные параметры, а их объединение позволяет системе более точно и надежно выявлять мошеннические действия.

В рамках системы противодействия банковским мошенничествам, разработанной компанией ООО «Компас Плюс» транзакции определены как потоки данных. Поток данных – это схема финансовых данных, имеющие свои поля с примитивными типами данных (число, строка, дата), например, время возникновения транзакции или логин пользователя. Потоки данных в

системе определяются банком, которому поставляется программный продукт.

В рамках текущей работы статическое рисковое правило состоит из следующих элементов:

- 1) условия срабатывания – итоговое логическое выражение, по которому проверяется удовлетворяет ли транзакция заданным критериям;
- 2) параметров, которые могут использоваться в условии срабатывания;
- 3) потока данных, в который возвращается результат работы правила;
- 4) действий, необходимых сделать после срабатывания рискового правила, например, вызов определенной функции в базе данных;
- 5) результата правила, определяющий как реагировать на совершенную транзакцию.

У правила есть несколько возможных результатов при срабатывании:

- 1) Frictionless – аутентификация держателя карты без дополнительных проверок;
- 2) Challenge – требуется дополнительная аутентификация;
- 3) Decline – отклонение транзакции;
- 4) Alert – генерация оповещения для сотрудников безопасности для дальнейшего расследования и принятия необходимых мер.

Все элементы рискового правила, кроме условия срабатывания, являются его свойствами. Правило может иметь несколько версий. Каждая версия полностью независима и может состоять из разных элементов правила, описанных выше. Версионирование правила помогает для внесения изменений для тех правил, которые находятся в эксплуатации и не могут быть изменены. Тогда создание новой версии правила поможет в разработке и тестировании новых изменений. У версии правила есть свой жизненный цикл, описанный в таблице 1.

Таблица 1 – Жизненный цикл версии рискового правила

Состояние	Описание	Работа с реальными данными	Возможность редактирования
InDesign	Версия правила находится в режиме разработки	Нет	Да
Loading	Переходное состояние из режима разработки в режим эксплуатации	Нет	Нет
Production	Версия правила находится в режиме эксплуатации	Да	Нет
Archive	Переходное состояние из режима эксплуатации в режим разработки	Нет	Нет

Только одна версия правила может находиться в режиме эксплуатации. Если версия правила уже находится в режиме эксплуатации, то при вводе другой версии этого правила в режим эксплуатации, предыдущая версия автоматически перейдет в режим разработки. При возникновении ошибки в переходных состояниях будет выполнен сброс до предыдущего состояния.

Веб-приложение – это программное обеспечение, которое запускается в веб-браузере. Работа веб-приложения осуществляется с помощью клиент-серверной технологии, где клиентом является веб-браузер, а в качестве сервера выступает веб-сервер. Клиент чаще всего выступает в роли пользовательского интерфейса, с помощью которого пользователь взаимодействует с сервером с помощью методов HTTP запросов [5].

В настоящее время набирают популярность одностраничные веб-приложения. Данный вид веб-приложений загружается на одну HTML страницу, а элементы динамические отображаются. Логика пользовательского интерфейса выполняется преимущественно в веб-браузере, а взаимодействие с веб-сервером происходит через веб-API. При этом значительно повышается скорость работы приложения, так как при каждом изменении содержимого страницы она не перезагружается [2].

Разрабатываемое веб-приложение представляет из себя конструктор, для создания, редактирования, версионирования и тестирования статических рисков правил в системе противодействия банковским мошенничествам.

1.2. Обоснование выбора средств реализации

Для разработки веб-приложения будет выбран веб-фреймворк Blazor WebAssembly. Это фреймворк для создания одностраничных интерактивных веб-приложений с использованием .NET через технологию WebAssembly. WebAssembly представляет собой новый открытый формат байт-кода, который исполняется в современных браузерах. Он позволяет запускать код, написанный на таких языках, как C, C++, C#, Rust, преобразованный в низкоуровневые инструкции ассемблера, и использовать его в вебе [8]. Этот формат отличается компактностью, высокой производительностью, близкой к нативной, и может работать в сочетании с JavaScript [9, 17]. Blazor использует концепции, схожие с фреймворками JavaScript, такими как Angular или React [7]. Он обрабатывает взаимодействие с пользователем и отображает обновления интерфейса в компонентах. Компоненты создаются в файлах с расширением «.razor» и включают разметку HTML и код на C# [3, 4].

Этот фреймворк был выбран по следующим причинам:

- 1) возможность разработки веб-приложений с использованием инструментов и технологий, реализованных на языке программирования C#;
- 2) большая часть функционала уже встроена в фреймворк, что упрощает разработку и ускоряет процесс внедрения;
- 3) производительность приближена к нативному коду;
- 4) основной бэкенд-сервис написан на фреймворке ASP.NET Core, что упрощает взаимодействие между клиентом и сервером;
- 5) требование со стороны компании ООО «Компас Плюс».

1.3. Сравнительный обзор аналогов

В ходе выполнения выпускной квалификационной работы были выделены системы, которые являются аналогами для разрабатываемого веб-приложения: Конструктор правил в системе Smart Fraud Detection [16] и конструктор правил в системе FICO Falcon Fraud Manager [12].

Конструктор правил в системе Smart Fraud Detection

Система Smart Fraud Detection предназначена для автоматизации деятельности сотрудников финансовых организаций по выявлению и предотвращению мошеннических транзакций в различных каналах обслуживания клиентов.

Конструктор правил используется для создания новых правил или корректировки параметров уже существующих. Создание или редактирование условия срабатывания правила реализован с помощью специальной веб-формы. На рисунке 1 показан пример пользовательского интерфейса конструктора.

Панель параметров правила: **Срок (в днях) с момента первого появления поставщика услуг интернета у Клиента** [Закреть]

Общие сведения

Название правила *
Срок (в днях) с момента первого появления поставщика услуг интернета у Клиента
Состояние: **Рабочее**
Номер: 95
Создано: 16.12.2020 17:57 admin Иванов Иван Иванович
Изменено: 16.12.2020 17:57 admin Иванов Иван Иванович

Комментарий

Описание

Настройки правила

Категория *
Правило
Тип проверки *
ANTIFRAUD
Приоритет *
9

Свернуть

Рисунок 1 – Конструктор правил Smart Fraud Detection

Пользователь может редактировать различные параметры правила: описание, комментарий, действия при срабатывании, параметры, приоритет, тип транзакции и т.д.

Для рискованных правил определены три состояния:

- 1) настройка – правило находится в режиме разработки;
- 2) рабочее – работающее правило, которое недоступно для редактирования;
- 3) тестовое – правило работает на промышленных данных.

В веб-приложении отсутствует возможность создание или редактирование условия срабатывания правила через код, а уровень вложенности

условий в веб-форме ограничен 5, что накладывает ограничения. Реализован механизм копирования, экспорта и импорта правил, но отсутствует возможность удаления правил.

В конструкторе не поддержано создание нескольких версий одного и того же правила, что может вызвать неудобство, при необходимости обновлении старого правила новым условием срабатывания или изменения другим параметров, так как придется на период внесения изменений вывести его из рабочего состояния.

Присутствует возможность тестирования правил на тестовых данных, но отсутствует механизм их создания и привязки к правилу через веб-приложение, что делает невозможным тестирование правила в процессе разработки и настройки.

Конструктор правил в системе FICO Falcon Fraud Manager

Конструктор правил FICO Falcon Fraud Manager – это мощный инструмент, который позволяет пользователям создавать, модифицировать и управлять рисковыми правилами, направленными на предотвращение мошенничества. Он предоставляет гибкие возможности для настройки рискованных правил, что делает его удобным для применения в различных финансовых учреждениях.

Конструктор правил FICO Falcon позволяет пользователям создавать правила с учетом различных условий и критериев:

- 1) множество условий – можно создать правила, которые проверяют несколько аспектов транзакции одновременно, например, сумму операции, тип карты, местоположение и тип клиента;
- 2) логика правил – позволяет использовать логические операторы, такие как AND, OR, чтобы создавать более сложные и многогранные условия для проверки транзакций;
- 3) поддержка пользовательских скриптов – для более сложных случаев можно использовать кастомные скрипты.

Конструктор правил оснащен удобным графическим интерфейсом, который облегчает процесс создания и настройки:

- 1) визуальное представление – процесс создания и редактирования правил интуитивно понятен благодаря визуальным инструментам;
- 2) предварительный просмотр – позволяет увидеть, как будут выглядеть правила до их применения, что помогает избежать ошибок в их настройке;
- 3) тестирование – пользователи могут тестировать созданные правила на исторических данных, чтобы увидеть, как они повлияют на транзакции и настроить их для оптимальных результатов.

Конструктор правил включает в себя функциональность для моделирования различных сценариев мошенничества, что позволяет увидеть, как правило будет работать на реальных или имитированных данных, прежде чем применять его в реальной эксплуатации. Данная программная система недоступна на территории Российской Федерации.

Выводы по первой главе

В первой главе была изучена и описана предметная область, для которой разрабатывается веб-приложение, были рассмотрены и выбраны средства для реализации приложения, а также проанализированы аналоги к разрабатываемой системе. Анализ аналогов показал, что FICO Falcon Fraud Manager недоступен на территории Российской Федерации, Smart Fraud Detection не обладает достаточной функциональностью: нет возможности создания нескольких версий одного правила, нет возможности создания тестовых финансовых транзакций для рискового правила. Оба аналога не предоставляют возможность для изменения условия срабатывания рискового правила с помощью редактора кода в пользовательском интерфейсе. В связи с этим разработка веб-приложения «Конструктор правил для автоматического выявления подозрительных транзакций в системе противодействия банковским мошенничествам» является актуальной.

2. ПРОЕКТИРОВАНИЕ

2.1. Функциональные и нефункциональные требования

В ходе проектирования веб-приложения были определены функциональные и нефункциональные требования, которым должна соответствовать проектируемая система.

Функциональные требования

Разрабатываемое веб-приложение должно удовлетворять следующим функциональным требованиям.

1. Система должна предоставлять пользователю возможность просматривать, создавать, изменять и удалять версии рисков правил.
2. Система должна предоставлять пользователю возможность помещать правило в архив для возможности восстановления.
3. Система должна предоставлять пользователю возможность написания условия срабатывания рисков правила с помощью редактора кода с вспомогательными элементами, как подсветка синтаксиса, подсказки для автозаполнения, отображение информации об объекте при наведении.
4. Система должна предоставлять пользователю создание условия срабатывания рисков правила с помощью специальной веб-формы «мастер создания правила».
5. Система должна предоставлять пользователю возможность изменять свойства версии рисков правила.
6. Система должна предоставлять пользователю возможность отображать сравнение двух версий одного рисков правила с подсветкой различий между исходными кодами и свойствами.
7. Система должна предоставлять пользователю возможность запускать версию правила в эксплуатацию и убирать ее из эксплуатации.
8. Система должна предоставлять пользователю возможность создавать тестовые данные (песочницы) для тестирования работы версии рисков правила.

9. В системе должен присутствовать механизм импорта/экспорта версий рисков правил с помощью файлов в формате «.json».

10. Система должна предоставлять пользователю возможность просмотра информации о потоках данных, которые представляют из себя схемы финансовых транзакций.

11. Система должна предоставлять механизм для импорта/экспорта потоков данных с помощью файлов в формате «.json».

12. В системе должна присутствовать аутентификация пользователей.

13. В системе должна присутствовать авторизация пользователей.

Нефункциональные требования

Разрабатываемое веб-приложение должно удовлетворять следующим нефункциональным требованиям.

1. Веб-приложение должно быть реализовано с помощью фреймворка Blazor webassembly [14, 17].

2. Веб-приложение должно быть написано с использованием языка разметки HTML, каскадных таблиц стилей CSS и языков программирования javascript и C#.

3. Редактора кода в веб-приложении должен быть реализован с помощью инструмента Monaco Editor [15].

4. Аутентификация пользователей должна быть реализована с помощью механизма openid Connect с использованием сервиса авторизации Keycloak [10].

5. Авторизация пользователей должна быть реализована с помощью получения их прав из бэкэнд сервиса системы противодействия банковским мошенничествам.

6. Контейнеризация веб-приложения должна быть реализована с помощью Docker [11].

2.2. Диаграмма вариантов использования

Для проектирования системы был использован язык графического описания UML. В соответствии с требованиями была построена диаграмма вариантов использования, представленная на рисунке 2.

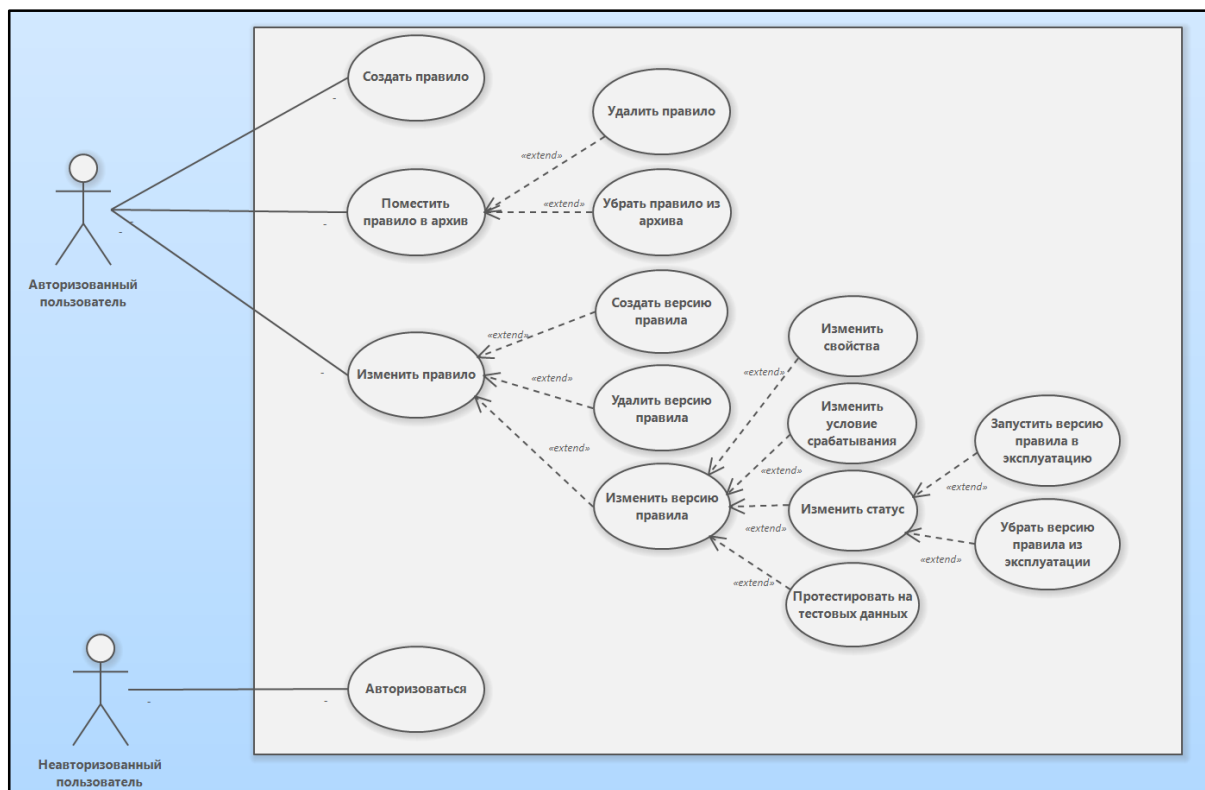


Рисунок 2 – Диаграмма вариантов использования

С системой взаимодействуют два актера: авторизованный и неавторизованный пользователи. Неавторизованный пользователь может только авторизоваться в системе, чтобы ему стал доступен основной функционал. Авторизованный пользователь может создать, поместить в архив (после чего удалить полностью или убрать из архива) или изменить правило. Чтобы изменить правило пользователь может создать, удалить или изменить версию правила. Изменение версии правила включает в себя несколько вариантов использования: изменить свойства, изменить условие срабатывания, изменить статус и протестировать на тестовых данных. Чтобы изменить статус пользователь может запустить версию правила в эксплуатацию, чтобы отслеживать и обрабатывать реальные финансовые транзакции или убрать версию правила из эксплуатации для внесения изменений, описанных выше.

Для тестирования версии правила на тестовых данных пользователь может создать специальную песочницу, которая непосредственно связана с самой версией и наполнить ее необходимыми данными, после чего может запустить процесс тестирования и посмотреть итоговые результаты.

2.3. Проектирование архитектуры системы

Система является клиент-серверным приложением. Для визуального представления архитектуры системы была построена диаграмма размещения UML, представленная на рисунке 3. На диаграмме представлена информация о связи между компонентами системы и протоколах, по которым они взаимодействуют.

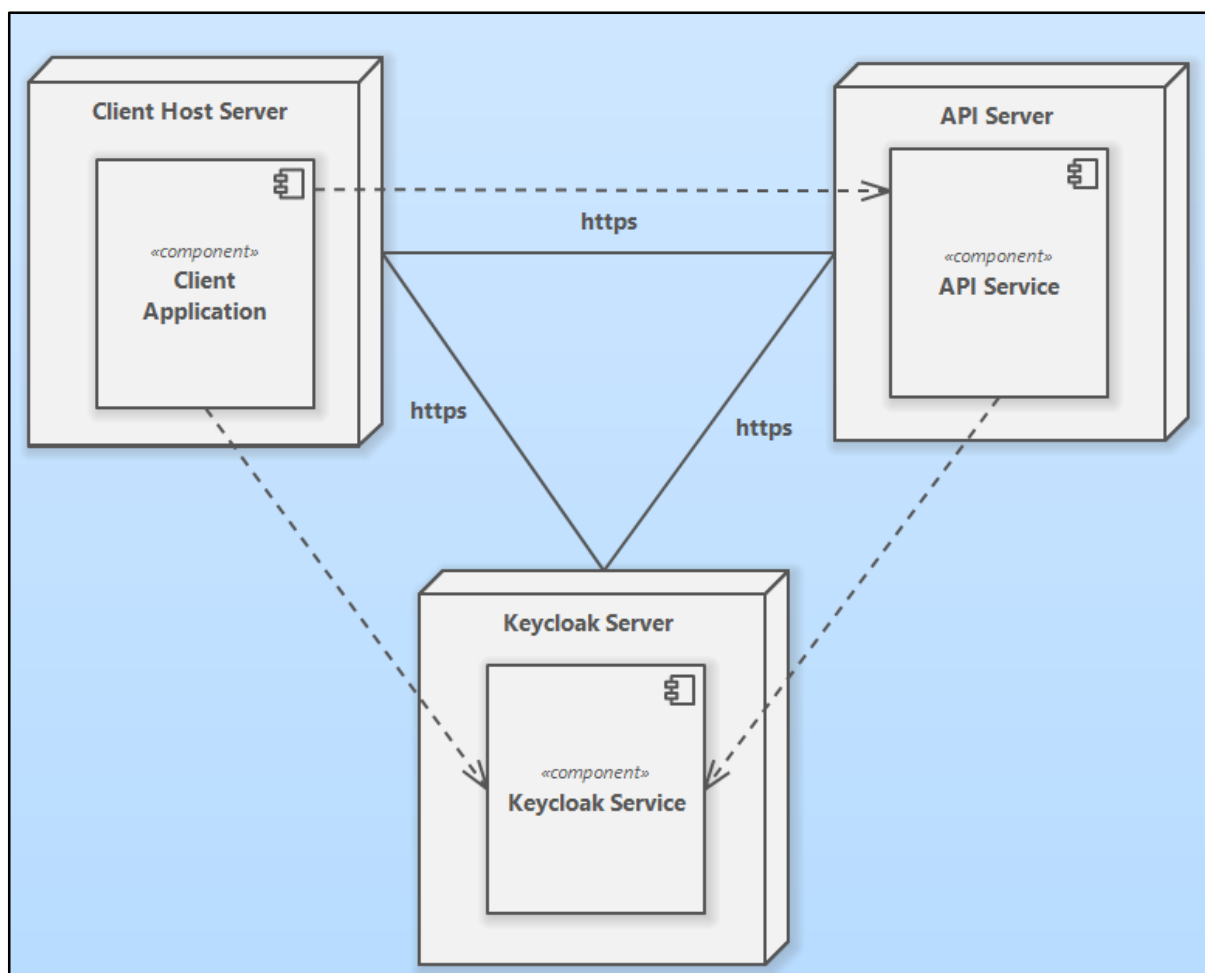


Рисунок 3 – Диаграмма размещения UML

Система состоит из 3 узлов: Client Host Server, API Server и Keycloak Server. Каждый узел взаимодействует с другими с помощью безопасного протокола передачи данных HTTPS.

Client Host Server

Представляет из себя сервер, на котором развернуто клиентское приложение. Компонент Client Application – клиентская часть веб-приложения, где с помощью веб-интерфейса пользователь взаимодействует с другими элементами системы. С помощью клиентского приложения пользователь авторизуется в системе, отправляя необходимые данные на сервер авторизации Keycloak Server, и после этого получая необходимые токены доступа. Данная информация используется клиентским приложением для отображения или скрытия различных компонент веб-интерфейса, а также отправляется в запросах к API серверу, для получения необходимых данных.

API Server

Представляет из себя сервер, на котором развернут бэкэнд-сервис для получения и обработки данных. API взаимодействует с клиентским приложением, предоставляя запрашиваемую информацию, а также выполняет аутентификацию и авторизацию пользователей с помощью токена доступа, который клиентское приложение отправляет в запросах. Используя токен доступа, API сервер проверяет его подлинность с помощью Keycloak. API сервис может обращаться к базе данных для изменения или извлечения данных, необходимых клиентскому приложению, например, информации о правилах и потоках данных.

Keycloak Server

Представляет из себя сервер, на котором развернут сервис аутентификации и авторизации Keycloak. Он обеспечивает проверку учетных данных пользователей, выдачу токенов доступа, управление ролями и правами, а также интеграцию с клиентским приложением и API сервисом для контроля доступа к защищенным ресурсам.

2.4. Диаграмма классов рискового правила

В результате анализа предметной области и функциональных требований была построена диаграмма классов UML (рисунок 4), отображающая классы, их поля и связи между ними, из которых состоит структура рискового правила.

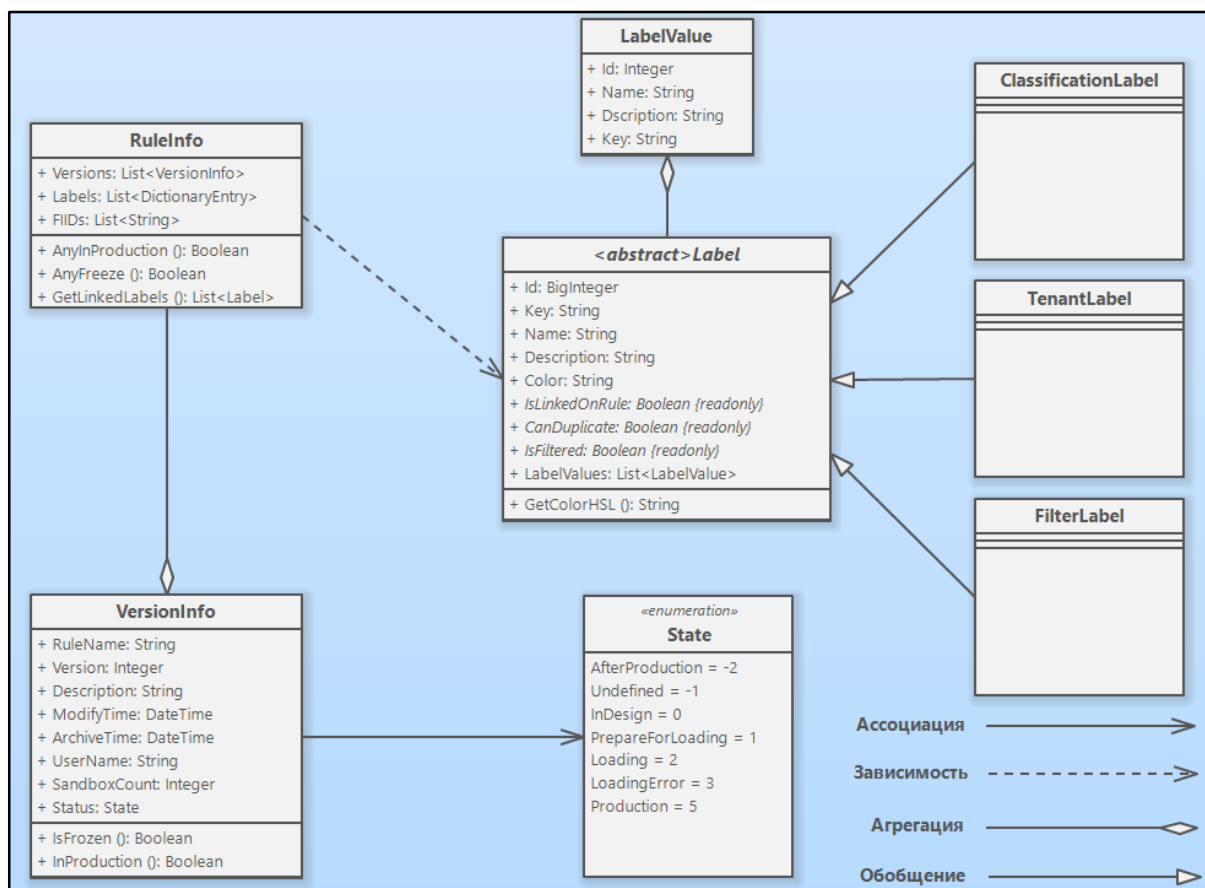


Рисунок 4 – Диаграмма классов рискового правила

Класс `VersionInfo` хранит общую информацию о версии правила. Перечисление `State` хранит различные статусы версии рискового правила. Абстрактный класс `Label` хранит информацию о метке для правила, с помощью которой удобно фильтровать и классифицировать различные рисковые правила. Класс `LabelValue` хранит информации о значении метки. Класс `RuleInfo` хранит информацию о всех версиях рискового правила, а также метках и финансовых институтах, назначенных на него. Класс `VersionInfo` связан с классом `RuleInfo` через связь агрегация и использует перечисление `State`, класс `RuleInfo` зависит от класса `Label`. Класс `LabelValue` связан с

классом `Label` через связь агрегация. Класс `Label` является родительским классом для классов: `ClassificationLabel`, `TenantLabel`, `FilterLabel`.

Класс `VersionInfo` состоит из следующих полей:

- 1) поле `RuleName` является строковой переменной и хранит информацию о имени правила;
- 2) поле `Version` является целочисленной переменной и хранит информацию о версии правила;
- 3) поле `Description` является строковой переменной и хранит информацию об описании версии правила;
- 4) поле `ModifyTime` хранит дату и время последнего изменения версии правила;
- 5) поле `ArchiveTime` хранит дату и время занесения правила в архив, может быть пустым;
- 6) поле `UserName` является строковой переменной и хранит имя пользователя, который сделал последнее изменение в версии правила;
- 7) поле `SandboxCount` является целочисленной переменной и хранит количество песочниц версии правила;
- 8) поле `Status` хранит информацию о статусе версии правила.

В данном классе присутствуют два метода `IsFrozen` и `InProduction`, возвращающие булевый результат о том, заморожена ли версия правила (не в режиме разработки) и находится ли в эксплуатации соответственно. Если результат истинный, то такая версия правила доступна только для чтения. В данных методах для проверки используется поле `Status`.

Перечисление `State` состоит из следующих значений:

- 1) `AfterProduction` – версия правила в процессе удаления версии из режима эксплуатации;
- 2) `Undefined` – неопределенное состояние версии правила;
- 3) `InDesign` – версия правила находится в режиме разработки и доступна для редактирования;

- 4) `PrepareForLoading` – состояние версии правила, перед запуском процесса для перехода в режим эксплуатации;
- 5) `Loading` – процесс перехода версии правила в режим эксплуатации;
- 6) `LoadingError` – статус версия правила, после возникновения ошибки в процессе перехода в режим эксплуатации;
- 7) `Production` – версия правила находится в режиме эксплуатации.

Класс `Label` состоит из следующих полей:

- 1) поле `Id` является целочисленной переменной и хранит информацию об уникальном идентификаторе метки;
- 2) поле `Key` является строковой переменной и хранит информацию о ключе метки;
- 3) поле `Name` является строковой переменной и хранит информацию об имени метки;
- 4) поле `Description` является строковой переменной и хранит информацию об описании метки;
- 5) поле `Color` является строкой переменной и хранит информацию о Hex-коде цвета метки;
- 6) поле `IsLinkedOnRule` является абстрактным булевым полем и хранит информацию о том, можно ли назначить метку на правило;
- 7) поле `CanDuplicate` является абстрактным булевым полем и хранит информацию о том, может ли метка дублироваться с разными значениями в правиле;
- 8) поле `IsFiltered` является абстрактным булевым полем и хранит информацию о том, используется ли метка для фильтрации правил;
- 9) поле `LabelValues` является списком объектов `LabelValue` и хранит информацию о значениях метки.

Все 3 абстрактных поля переопределяются в классах-потомках `ClassificationLabel`, `FilterLabel`, `TenantLabel`. Метка, являющаяся

объектом класса `ClassificationLabel` может назначаться на правило, использоваться для фильтрации, но не дублироваться. Метка, являющаяся объектом класса `FilterLabel` не может назначаться на правило и дублироваться. Метка, являющаяся объектом класса `TenantLabel` может назначаться на правило, дублироваться с различными значениями и использоваться для фильтрации.

Метод `GetColorHSL` возвращает строковое значение цвета метки, который переводится из Hex-кода, хранящегося в поле `Color`, в формат HSL, основанный на оттенке, насыщенности и яркости для его отображения на пользовательском интерфейсе.

Класс `LabelValue` состоит из 4 полей: `Id` (уникальный идентификатор значения метки), `Name` (наименование значения метки), `Key` (ключ значения метки), `Description` (описание значения метки).

Класс `RuleInfo` состоит из 3 полей: `Versions` (список объектов версий правила `VersionInfo`), `Labels` (список классификационных меток, назначенных на правило), `FIIDs` (список финансовых институтов, назначенных на правило).

Методы `AnyFreeze` и `AnyProduction` возвращают булево значение, которое говорит о том, есть ли в правиле хотя бы одна замороженная версия или версия в эксплуатации соответственно. Метод `GetLinkedLabels` возвращает список объектов `Label`, формирующийся из полей `Labels` и `FIIDs`.

2.5. Диаграмма классов версии рискового правила

В результате анализа предметной области и функциональных требований была разработана диаграмма классов, отображающая структуру версии рискового правила. В ней отображены классы, их поля и связи между ними. Диаграмма разрабатывалась с помощью языка графического описания UML, представленная на рисунке 5.

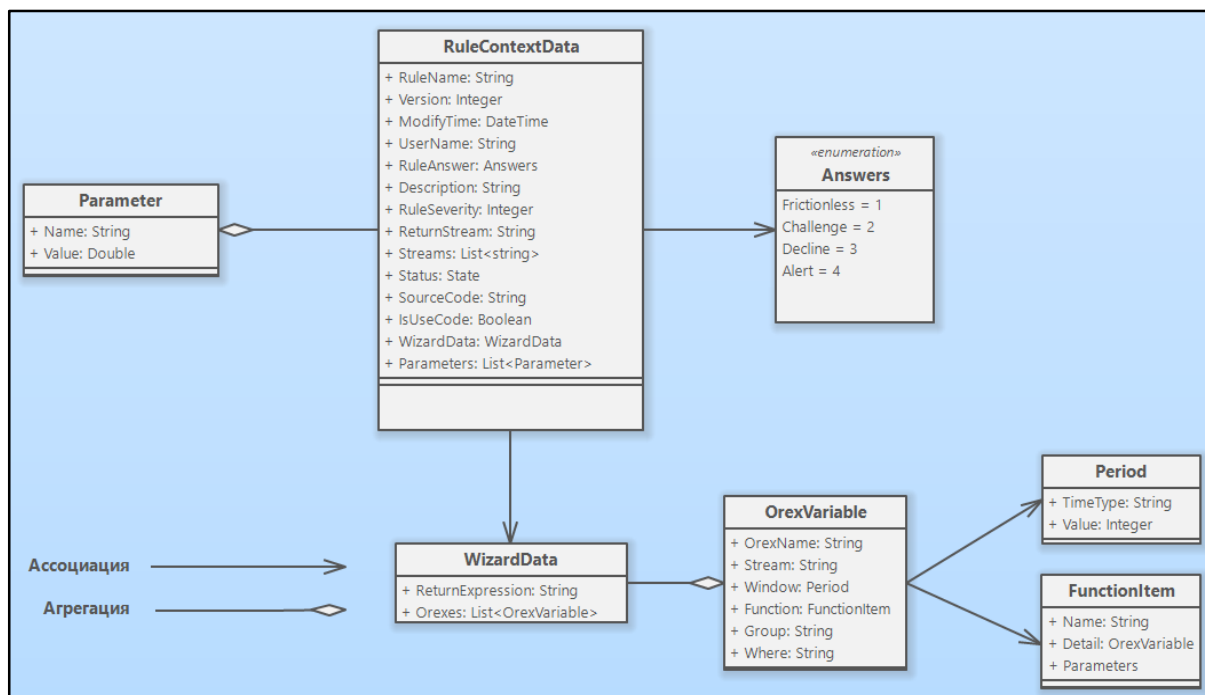


Рисунок 5 – Диаграмма классов версии рискового правила

Класс `RuleContextData` хранит информацию о версии рискового правила. Перечисление `Answers` хранит все возможные варианты реагирования после выполнения условия срабатывания правила, описанные в 1 главе. Класс `Parameter` хранит информацию о параметре в версии правила. Класс `WizardData` хранит данные «Мастера создания правила». Класс `OrexVariable` хранит данные об агрегате версии рискового правила. Класс `Period` хранит информацию о времени, за которое агрегируются данные потока, используемые в версии рискового правила. Класс `FunctionItem` хранит информацию об агрегатной функции, применяемой к данным потока.

Класс `RuleContextData` связан с классами `Parameter` и `WizardData` с помощью связей агрегация и ассоциация соответственно. Класс `WizardData` связан с классом `OrexVariable` с помощью связи агрегация. Класс `OrexVariable` связан с классами `Period` и `FunctionItem` с помощью связи ассоциация. Перечисление `Answers` используется в `RuleContextData`.

Класс `RuleContextData` состоит из следующих полей.

1. Поле `rulename` является строковой переменной и хранит информацию об имени рискового правила.

2. Поле `Version` является целочисленной переменной и хранит версию рискового правила.
3. Поле `Description` является строковой переменной и хранит информацию об описании версии рискового правила.
4. Поле `modifytime` хранит информацию о дате и времени последнего изменения версии.
5. Поле `username` является строковой переменной и хранит логин пользователя, сделавшего последние изменения.
6. Поле `ruleanswer` хранит информацию о результате работы правила и определяет, как реагировать на транзакцию после выполнения условия срабатывания.
7. Поле `ruleseverity` является целочисленной переменной и хранит информацию о приоритете правила.
8. Поле `returnstream` является строковой переменной и хранит имя основного потока, данные которого будут анализироваться при работе правила и в который будет возвращаться результат работы правила.
9. Поле `Streams` является списком, хранящий имена потоков, используемые в версии рискового правила.
10. Поле `Status` хранит информацию о статусе версии правила, описанного выше.
11. Поле `sourcecode` является строковой переменной и хранит информацию об условии срабатывания правила.
12. Поле `isusecode` является булевой переменной и хранит информацию, как было создано условие срабатывания правила, а именно с помощью редактора кода или специальной веб-формы «Мастер создания правила».
13. Поле `wizarddata` является объектом одноименного класса и хранит данные «Мастера создания правила».
14. Поле `Parameters` является списком, который хранит данные о параметрах версии правила.

Класс `Parameter` состоит из 2 полей. Поле `Name` является строковой переменной и хранит имя параметра, поле `Value` является числовой переменной и хранит значение параметра. Параметр может использоваться в исходном коде версии рискового правила.

Класс `WizardData` состоит из 2 полей. Поле `ReturnExpression` является строковой переменной и хранит итоговое выражение, определяющее условие срабатывания правила. Поле `Orexes` является списком объектов `OrexVariable`, который хранит агрегаты, используемые в версии правила. Агрегаты – это выражения, которые работают с множеством значений и возвращают один результат.

Класс `OrexVariable` состоит из следующих полей:

- 1) поле `OrexName` является строковой переменной и хранит информацию об имени агрегата;
- 2) поле `Stream` является строковой переменной и хранит имя потока, данные которого используются в агрегате;
- 3) поле `Window` является объектом класса `Period` и хранит информацию о промежутке времени, по которому вычисляется агрегатная функция;
- 4) поле `Function` является объектом класса `FunctionItem` и хранит функцию, применяемую к данным потока;
- 5) поле `Group` является строковой переменной, которое хранит выражение, по которому группируются агрегируемые значения;
- 6) поле `Where` является строковой переменной, которое хранит выражение, по которому фильтруются агрегируемые значения.

Список потоков, которые могут быть использованы в агрегате состоит из тех, что используются в версии рискового правила. В агрегатное выражение попадают только те потоки, время создания которых попадает в интервал, вычисляемый от значения, которое вычисляется с помощью вычитания из текущего времени данных в поле `Window`, до текущего времени. В качестве функций для агрегирования будут использованы аналоги агрегатных

функций SQL, которые поддерживаются в системе противодействия банковским мошенничествам.

Класс `Period` состоит из 2 полей. Поле `TimeType` является строковой переменной и хранит информацию об единице измерения времени: секунды, минуты, часы, дни, месяцы. Поле `Value` является целочисленной переменной и хранит информацию о значении.

Класс `FunctionItem` состоит из 3 полей. Поле `Name` является строковой переменной и хранит информацию об имени функции. Поле `Detail` является строковой переменной и хранит информацию об описании функции. Поле `Parameters` является списком и хранит информацию о параметрах функции: имя параметра, описание параметра и тип данных параметра.

Выводы по второй главе

Во второй главе были определены функциональные и нефункциональные требования к программному продукту, на основе которых была построена диаграмма вариантов использования, определены основные актеры, приведены краткое описание взаимодействия актеров с системой. Для описания архитектуры веб-приложения была построена диаграмма размещения и описаны основные элементы архитектуры. Были разработаны диаграммы классов, которые определяют структуру рискового правила и его версий. Также было описано назначение классов, их поля и связи между классами.

3. РЕАЛИЗАЦИЯ

3.1. Программные средства реализации

Серверная часть веб-приложения была написана с помощью фреймворка ASP.NET Core на языке программирования C#. В качестве среды выполнения используется .NET 8 версии с долгосрочной поддержкой в течение 3 лет.

Клиентская часть веб-приложения была написана с помощью фреймворка Blazor WebAssembly, который используется языка программирования C# и JavaScript, язык разметки HTML и каскадные таблицы стилей CSS для создания одностраничных веб-приложений.

Аутентификация и авторизация пользователей была реализована с помощью механизма OpenID Connect с использованием JSON Web Token для защиты доступа к функционалу системы противодействия банковским мошенничествам. В качестве сервиса аутентификации и авторизации используется программный продукт Keycloak.

Для контейнеризации веб-приложения было использовано программное обеспечение Docker. С помощью Docker все необходимые библиотеки, зависимости и кодовая база были установлены в определенный Docker образ.

3.2. Реализация серверной части

Для размещения и обслуживания клиентского приложения, разработанного с использованием Blazor WebAssembly, была выбрана архитектура с использованием серверного приложения на базе ASP.NET Core. Серверное приложение в данном проекте выполняет функции, описанные ниже.

1. Хостинг статических файлов. Приложение Blazor WebAssembly компилируется в набор статических ресурсов, включающих HTML, JavaScript, CSS и сборки .NET. Серверная часть на ASP.NET Core обеспечивает доступ к этим ресурсам, реализуя передачу клиенту готового SPA-приложения.

2. Маршрутизация запросов. В случае использования способа обработки маршрутов, когда маршруты обрабатываются на стороне клиента, серверная часть обрабатывает запросы на все маршруты и возвращает клиенту начальный HTML-файл (index.html), позволяя Blazor WebAssembly корректно загрузиться и отработать маршрутизацию.

3. Безопасность и управление доступом. Хотя сервер в данном проекте не выполняет бизнес-логику и не взаимодействует напрямую с базой данных, он может обеспечивать базовые механизмы безопасности, такие как настройка политики CORS, HTTPS, редиректы, обработку ошибок и защиту от несанкционированного доступа.

3.2. Реализация клиентской части

Клиентская часть веб-приложения состоит из 3 главных составляющих: навигационная панель, интерфейс для работы с версией рискового правила и интерфейс для тестирования версии рискового правила. Ниже представлено подробное описание реализации всех 3 составляющих.

Навигационная панель

Навигационная панель используется для просмотра и редактирования различных рисковых правил, потоков данных и навигации пользователя между различными версиями правил. На рисунке 6 представлен скриншот всплывающего меню веб-приложения для работы с рисковыми правилами.

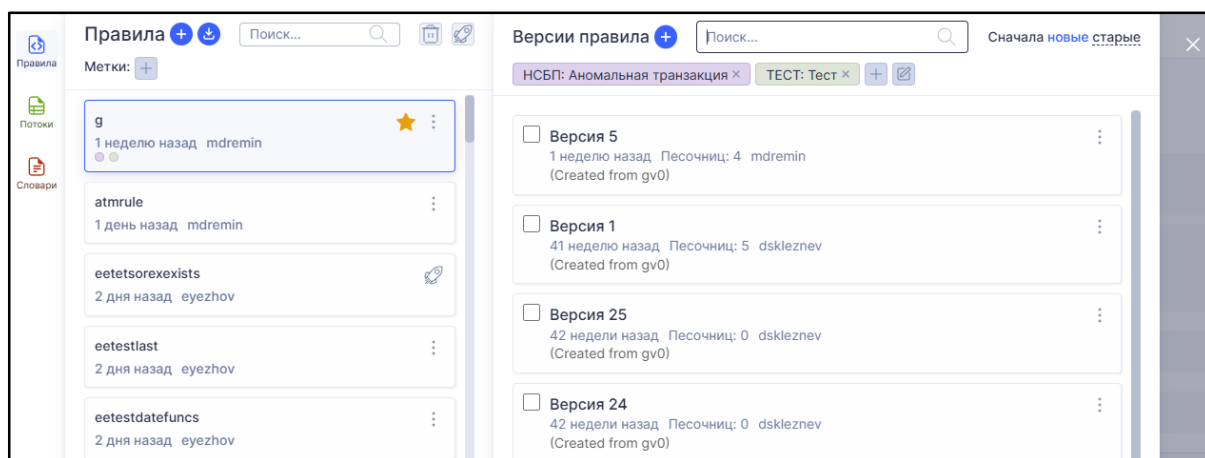


Рисунок 6 – Всплывающее меню рисковых правил

Для реализации данной веб-компоненты были использованы классы, описанные в диаграмме классов рискового правила. Основной функционал описан ниже.

1. Просмотр списка правил и их версий. Для фильтрации списка можно применять различные фильтры: метки, статус правил, поле ввода для поиска правил и их версий.
2. Создание нового правила. По умолчанию создается нулевая версия нового правила.
3. Импорт версий правил через файлы в формате «json».
4. Перемещение правила в корзину. В корзину можно перемещать только те правила, где все версии находятся в режиме разработки. В корзине можно либо полностью удалить правило, либо восстановить его.
5. Добавление правила в избранное.
6. Просмотр корзины правил. Для фильтрации используется тот же функционал, что и для списка правил.
7. Открытие версии правила. Для получения более подробной информации о версии правила пользователь может открыть ее в специальной вкладке.
8. Создание новой версии правила. Сюда включается возможность дублировать определенную версию правила, создать новое правило с другим названием на основе выбранной версии текущего правила.
9. Удаление версии правила. Удаляться могут только версии правила находящиеся в режиме разработки. При удалении последней версии правила, удаляется и само правило.
10. Экспорт версии правила в файл формата «json».
11. Назначение меток на правило. Может использоваться для классификации правил по определенному признаку, и также для назначения на правило определенного финансового института.

12. Сравнение исходного кода двух версий одного правила. Для этого пользователь может выбрать с помощью чекбоксов в панели версий две необходимых ему версии и нажать кнопку «Сравнить».

Правила в своей работе используют различные потоки данных (схемы финансовых транзакций), при этом потоков данных может быть большое количество и каждый поток может иметь большое количество полей. В связи с этим, была разработано всплывающее меню для работы с потоками данных, представленное на рисунке 7.

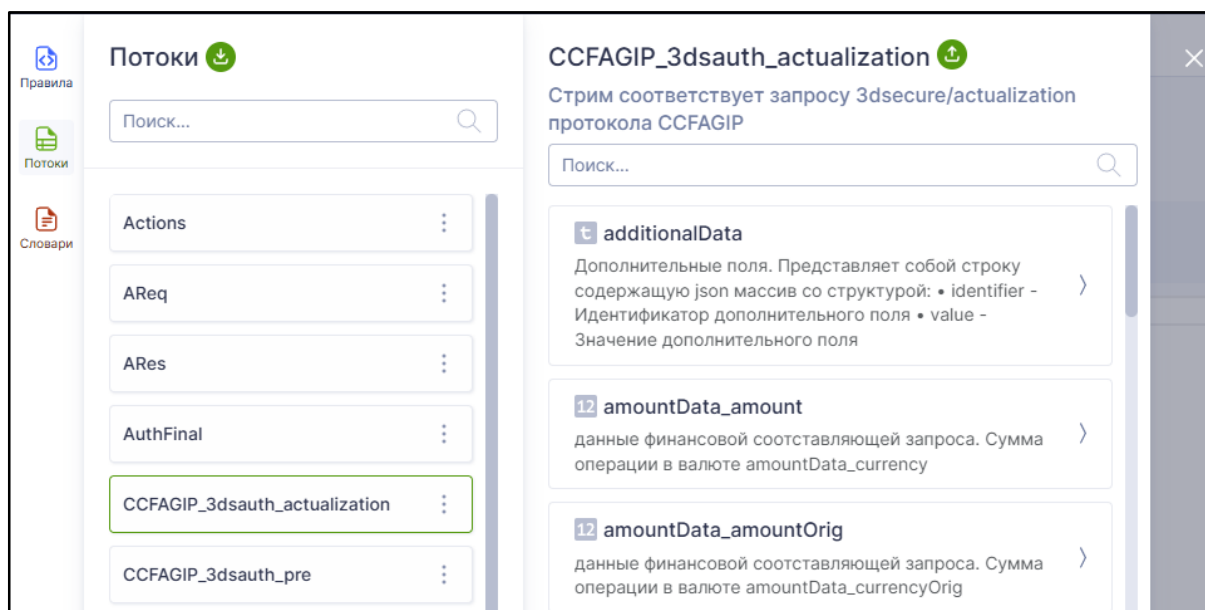


Рисунок 7 – Всплывающее меню потоков данных

Данное меню предназначено для просмотра потоков данных в системе. Каждый поток данных имеет свое уникальное наименование, описание и список полей. Имя поля в рамках потока данных состоит из уникального имени, описания и типа данных. Для фильтрации списка потоков данных и их полей используются поля ввода для поиска в верхних частях меню. Также поток данных можно импортировать с помощью «json» файла определенной структуры и экспортировать в файл того же формата. Если такой поток существует, то после импорта его данные обновляются данными из файла, иначе создается новый.

Редактор меток

Для классификации рисковых правил была разработана веб-форма для редактирования меток. Каждая метка состоит из уникального ключа, наименования, множества значений и цвета. Значение метки состоит из уникального ключа в рамках текущей метки и наименования. На рисунке 8 представлен веб-интерфейс для редактирования меток.

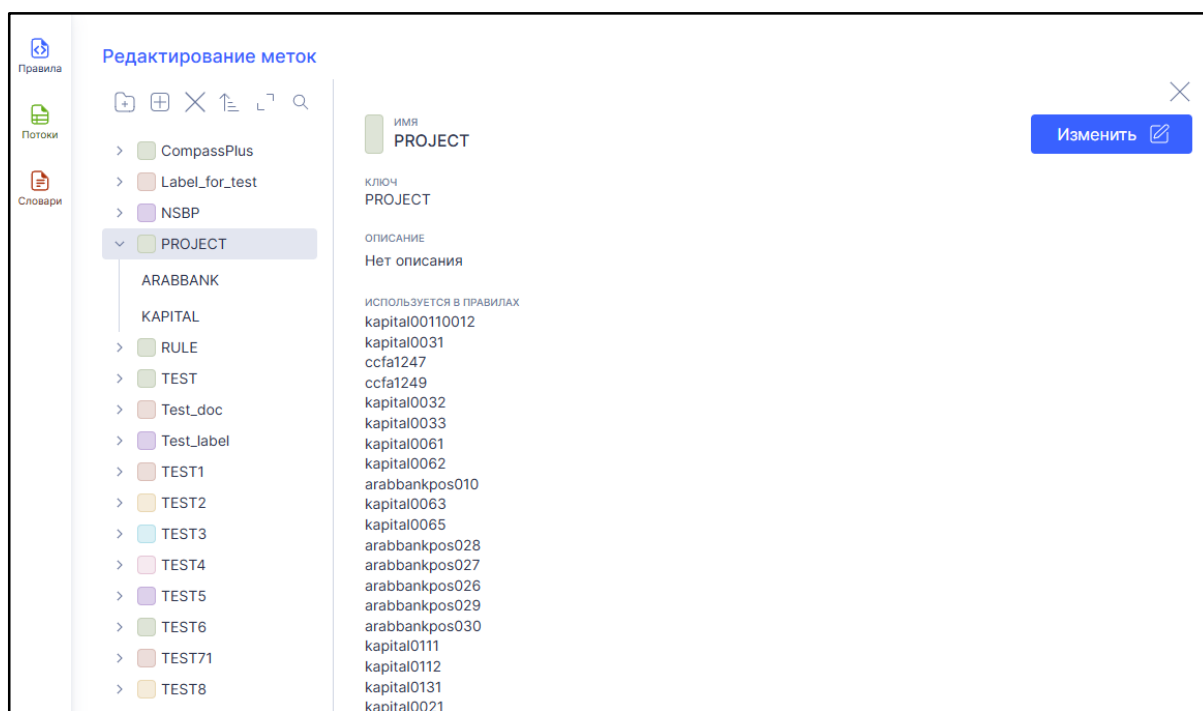


Рисунок 8 – Веб-интерфейс для редактирования меток

В редакторе меток реализован следующий функционал: просмотр списка меток и их значений, создание метки и ее значений, удаление метки и ее значений, сортировка списка меток по ключу, поиск меток и их значений в общем списке, просмотр информации о метке и ее значениях, редактирование метки, редактирование значения метки.

На одно рисковое правило можно назначить несколько классификационных меток, но в рамках одной метки может быть выбрано только одно значение. Метку, которая была назначена на рисковое правило нельзя удалить, и то значение, которое было выбрано тоже. Также при просмотре информации о метке или значении метки отображаются рисковые правила, на которые они назначены.

Веб-интерфейс для работы с версией правила

Веб-интерфейс для работы с версией правила состоит из 3 частей: область для просмотра и редактирования исходного кода версии (условия срабатывания), область для просмотра и редактирования свойств версии и область для тестирования версии (рисунок 9).

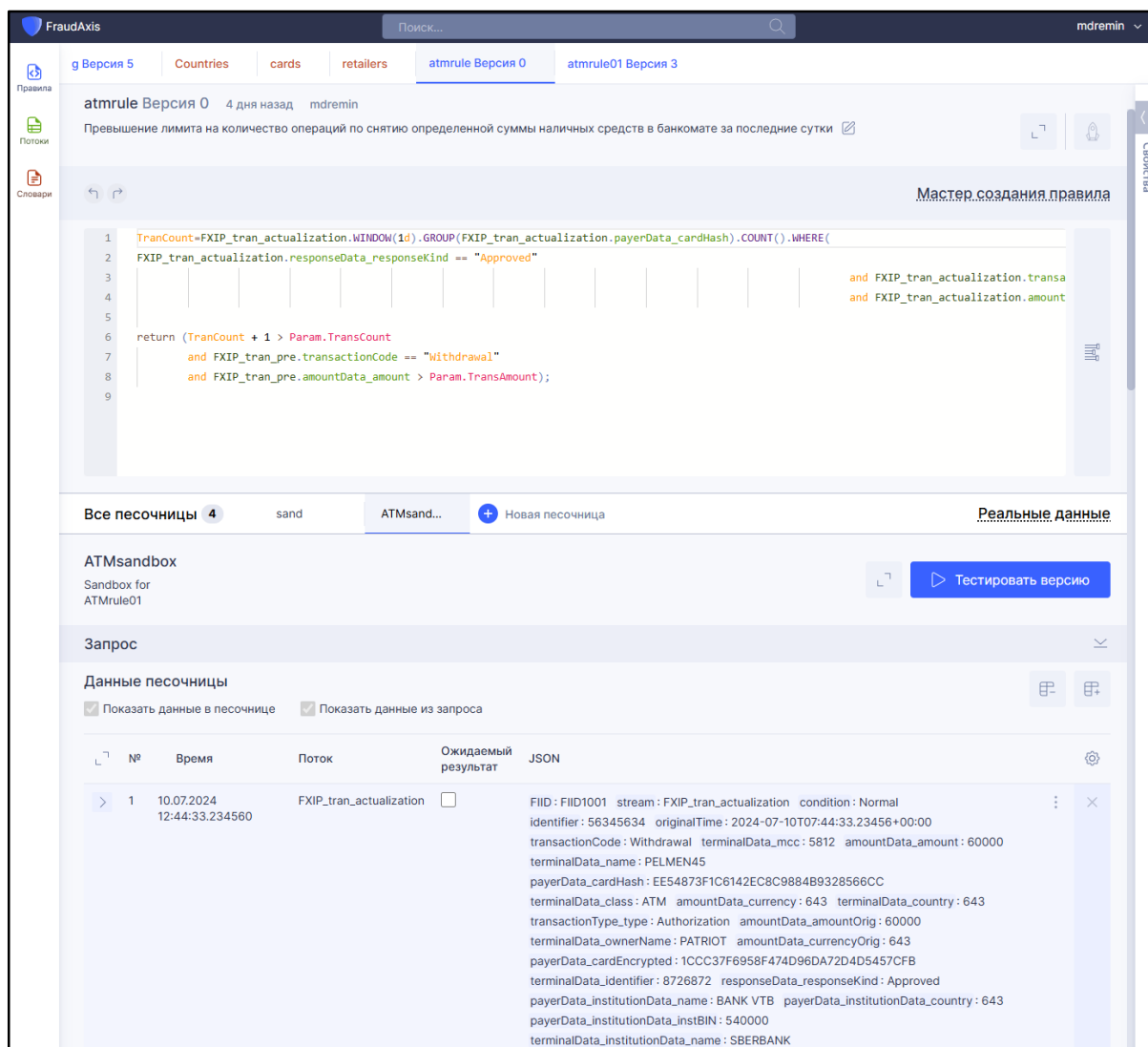


Рисунок 9 – Веб-интерфейс для работы с версией правила

Для редактирования исходного кода версии правила используется специально настроенный веб-редактор, для редактирования свойств правила используется боковая панель, расположенная в правой части страницы, для тестирования версии правила используются песочницы, где можно создавать и редактировать потоки данных. Также присутствует возможность тестирования версии правила на реальных данных.

Редактор кода

Редактор кода для создания исходного кода версии правила был реализован на основе редактора кода Monaco Editor [15]. На рисунке 10 представлена диаграмма активности UML, которая показывает взаимодействие пользователя с редактором.

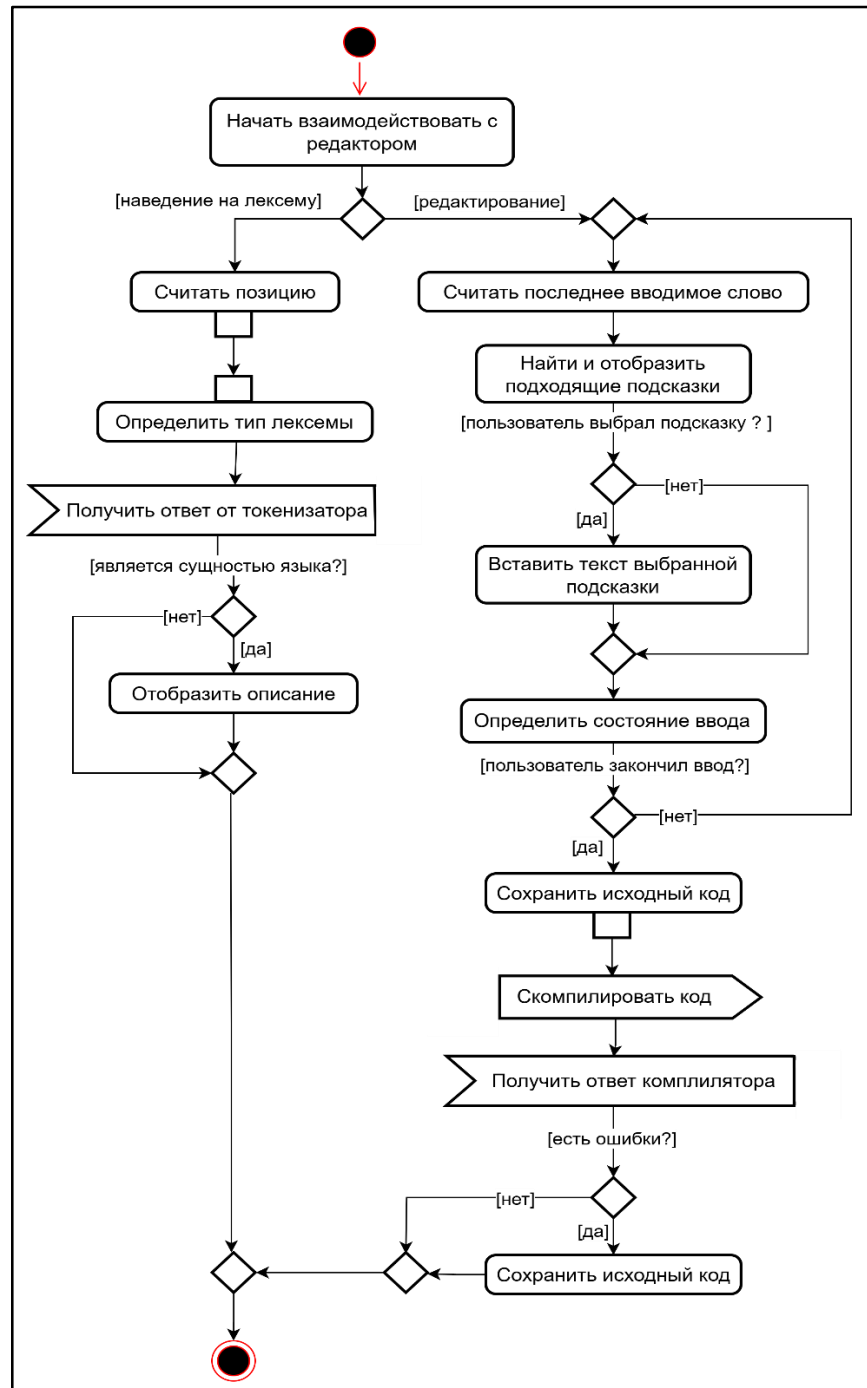


Рисунок 10 – Диаграмма активности работы редактора кода

Редактор представляет функционал и API для создания собственного языка, настройку подсветки синтаксиса, создание механизма подсказок и

автозаполнения при редактировании кода, создание механизма отображения описания различных сущностей языка, создание механизма отображения сигнатуры функции во время ее заполнения.

Для создания подсветки синтаксиса был использован специальный инструмент редактора Monarch, в котором используется токенизатор для определения правил на основе регулярных выражений, по которым подсвечиваются необходимые лексемы и передаются определенные сущности языка для их подсветки. В качестве языка программирования используется язык DSL, разработанный компанией ООО «Компас Плюс». Основные сущности языка передаются на клиентскую часть с помощью http запросов.

Для удобства пользователя реализован функционал для отображения подсказок, с помощью которых он может заполнять исходный код, функционал для отображения описания сущностей исходного языка при наведении курсора на них, функционал для отображения сигнатуры функции в момент ее заполнения параметрами. На рисунке 11 показан пример окна с подсказками для заполнения кода.

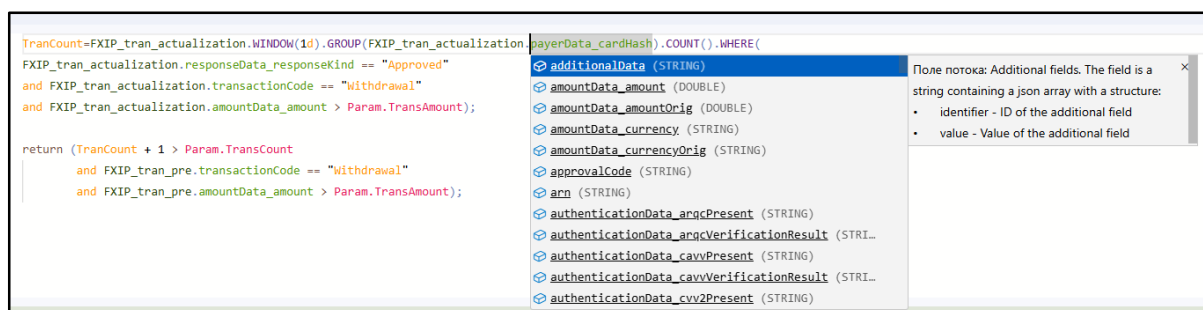


Рисунок 11 – Окно с подсказками для заполнения кода

Когда пользователь начинает вводить новые символы, редактор анализирует предыдущий текст и отображает необходимые подсказки. С их помощью пользователь может удобно заполнять исходный код и просматривать дополнительную информацию о них. Если последним словом является функция с параметрами, то в специальном окне возле нее отображается ее сигнатура (тип данных и список необходимых параметров). На рисунке 12 представлен пример окна для отображения сигнатуры функции.

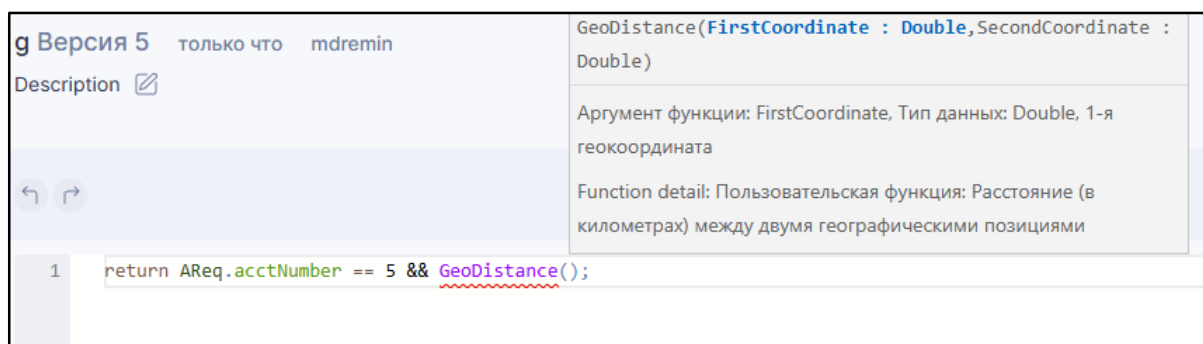


Рисунок 12 – Окно для отображения сигнатуры функции

В тот момент, когда пользователь заканчивает ввод, происходит сохранение исходного кода и его компиляция. Компиляция происходит с помощью http запроса, в котором отправляется исходный код версии правила. Ответом этого запроса является список, в котором указывается информация об ошибках в исходном коде. Ошибка состоит из описания, уровня серьезности и позиции в тексте. После компиляции ошибки подсвечиваются в виде красной волнистой линии, при наведении на которую показывается ее описание (рисунок 13).



Рисунок 13 – Подсветка и отображение ошибок компиляции

Для того, чтобы не нагружать API сервис запросами на компиляцию кода в момент, когда пользователь начинает ввод текста, включается таймер на 3 секунды, который сбрасывает пройденное время, если в течение этого времени пользователь продолжил вводить новые символы. Таким образом редактор считает, что пользователь закончил ввод только тогда, когда после последнего введенного символа прошло 3 секунды.

Панель свойств версии рискового правила

В правой части веб-интерфейса для работы с версией рискового правила была разработана панель, с помощью которой пользователь может просматривать и редактировать основные свойства версии рискового правила (рисунок 14).

The screenshot shows a web interface for managing risk rules. The main area displays a code editor for a rule named 'atmrule Версия 0'. The code logic involves checking transaction counts and transaction codes. Below the code editor is a table of sandbox environments. To the right, a sidebar titled 'Свойства' (Properties) contains various configuration options for the rule version, which are highlighted by a red box.

Свойства

Поиск...

Основные

Время модификации: 26.04.2025 17:37:40

Имя правила: atmrule

Версия: 0

Тип версии правила

Тип версии правила: RuleAnswer

Возвращаемое значение

Возвращаемое значение: Decline

Опции

Приоритет: 1

Компиляция

Параметры: Количество: 2

Проверка поля: Количество: 1

Подсказки компилятора

Возвращаемые значения: Количество: 1

Действие при сработке правила

Генерировать а...: False

Действие при с...: не задано

Вычисляемое поле

Вычисляемое поле: не задано

№	Имя	Время изменения	Последний тест	Изменено пользователем	Строк	Результат теста	Описание
1	sand	04.01.2025 11:42:49		mnovokreshenova	0		
2	sand02	04.01.2025 11:42:49		mnovokreshenova	0		
3	sand	04.01.2025 11:42:49		mnovokreshenova	0		
4	ATMsandbox	04.01.2025 11:42:49		mnovokreshenova	7		Sandbox for ATMRule01

Рисунок 14 – Панель свойств версии рискового правила

В данной панели указывается информация о имени правила, номере версии, времени последнего изменения, возвращаемом значении, приоритете, параметрах и основном потоке данных. Возвращаемое значение определяет в системе противодействия банковским мошенничествам, как реагировать на финансовую транзакцию (описано в 1 главе). Приоритет определяет степень важности возвращаемого значения (насколько результат работы правила приоритетнее относительно других). Параметры версии пра-

вила могут использоваться в виде констант в исходном коде (условии срабатывания). Основной поток данных определяет вид финансовой транзакции, данные которого будут анализироваться в работе рискового правила и в который будет возвращаться результат работы правила. Имя правила, номер версии и время изменения недоступны для редактирования.

Режим сравнения двух версий одного рискового правила

Дополнительно в веб-приложении реализован функционал для сравнения двух версий одного рискового правила. Сравнение происходит между исходным кодом версий правила и их свойствами. На рисунке 15 представлен скриншот веб-интерфейса сравнения двух версий.

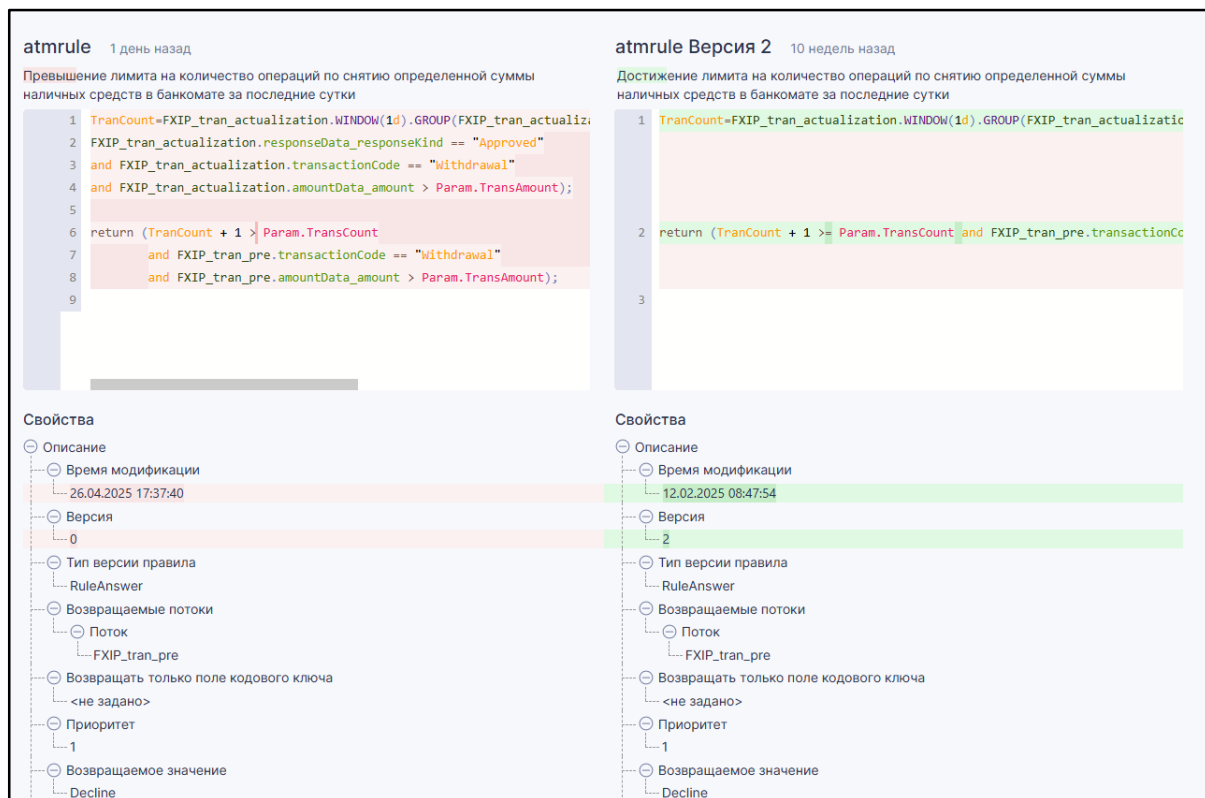


Рисунок 15 – Сравнение версий одного рискового правила

Для сравнения исходного кода используется специальный режим редактора «Diff Editor», который показывает оригинальный и измененный текст, а также подсвечивает различия между ними (добавления, удаления, изменения). Редактирование исходного кода в данном режиме запрещено.

Сравнение свойств версий правила реализовано с помощью сравнения узлов двух древовидных структур, где каждый узел является свойством версии рискованного правила. В левой части отображаются свойства исходной версии, а в правой измененной. При нахождении различий левый узел подсвечивается красным цветом, а правый зеленым. Редактирование свойств версий запрещено.

Мастер создания правила

«Мастер создания правила» - режим создания/описания версии правила без написания исходного кода. Для описания правила используются графические элементы интерфейса (рисунок 16).

Рисунок 16 – Мастер создания правила

В данном режиме объединены механизмы для редактирования исходного кода и изменения свойств версии правила. Редактирование исходного кода реализовано с помощью специальных элементов веб-интерфейса, где пользователь может выбрать необходимые сущности языка DSL (потoki

данных, поля и функции потоков данных, операторы, константы). Для компиляции кода используется тот же http запрос, что и в редакторе кода.

В исходных кодах многих рисковых правил могут использоваться агрегатные выражения, структура и назначения которых подробно описаны во 2 главе. Например, требуется рисковое правило, которое будет определять допустимый лимит операций по снятию наличных средств с карты определенной суммы за последние сутки. Такое рисковое правило должно учитывать множество транзакций в течение определенного периода времени. Для реализации функционала редактирования агрегатных выражений была реализована веб-форма, представленная на рисунке 17.

Агрегат будет сохранен, если:

- ✓ задано название агрегата
- ✓ выбран поток агрегата
- ✓ выбран период агрегации
- ✓ выбрана агрегирующая функция и аргумент (при наличии)
- ✓ задано условие группировки

Редактирование агрегата

Название:

Поток:

Период:

Смещение:

Функция:

Группировать по:

Если:

- ==
- and ==
- and >

☐ Проверка аномальности

Рисунок 17 – Редактор агрегатного выражения

Для создания агрегатного выражения указывается поток данных, данные которого будут агрегировать, период, агрегатная функция (например, подсчет количества), поле потока данных для группировки и условие для фильтрации агрегируемых значений. Период вычисляет окно времени, по которому агрегатное выражение анализирует только транзакции с подходящим временем создания. Например, если нужно проанализировать транзакции за последние сутки, то в качестве периода указывается 1 день.

Запуск версии рискового правила в эксплуатацию

Для того, чтобы рисковое правило работало с реальными финансовыми транзакциями, необходимо перевести его в эксплуатационный режим. На рисунке 18 представлена диаграмма активности UML, в которой отображен процесс перехода версии правила в эксплуатационный режим.

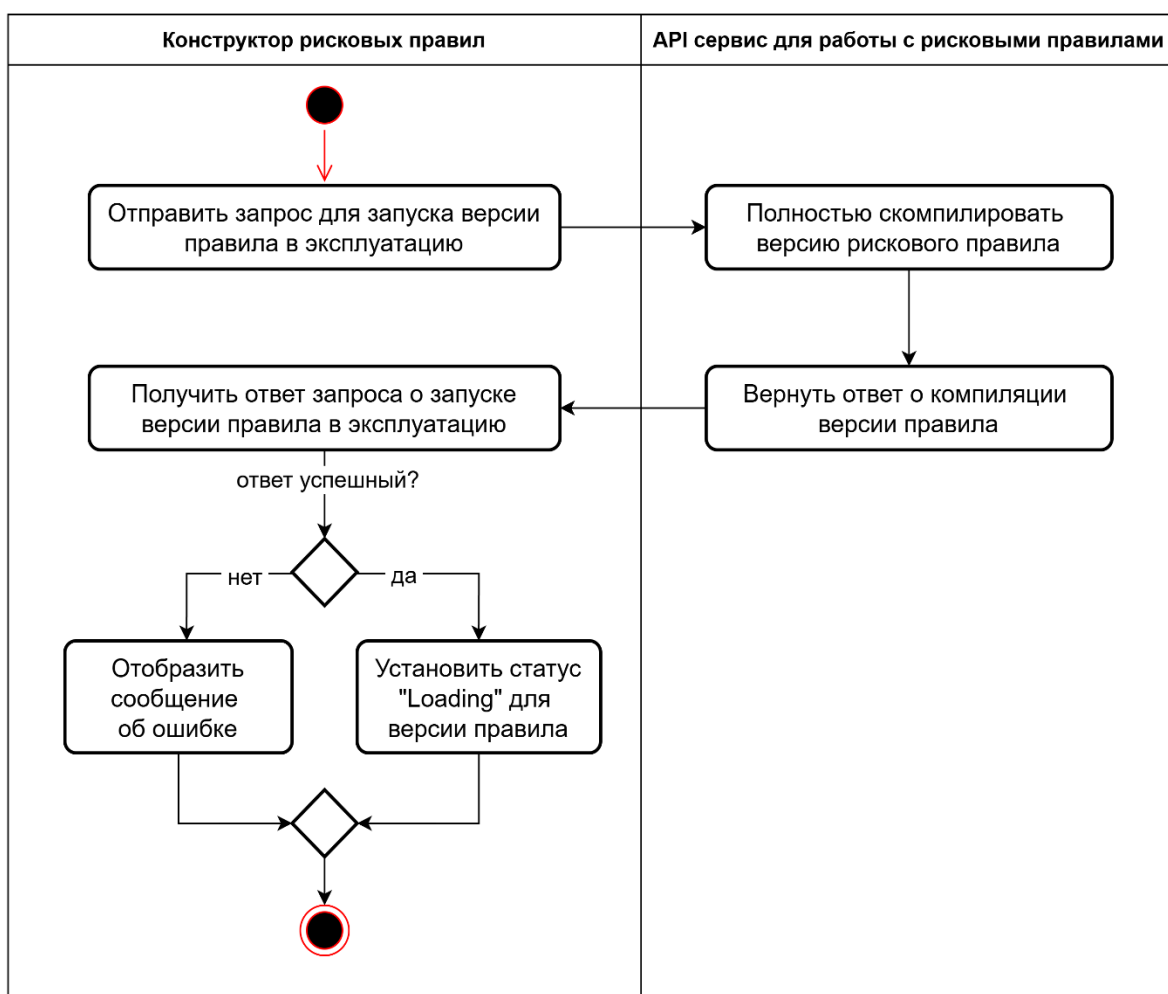


Рисунок 18 – Процесс запуска версии правила в эксплуатацию

После того, как пользователь запустит процесс перехода версии рискового правила в эксплуатацию (нажмет на иконку ракеты в правом верхнем углу страницы), отправится запрос для полной компиляции версии рискового правила. Полная компиляция версии правила – это процесс сохранения данных версии рискового правила, компиляции исходного кода, создание и заполнение необходимых структур в хранилище данных. Если в версии правила используются агрегатные выражения, то в хранилище данных будут созданы необходимые таблицы, которые будут хранить в себе данные совершенных финансовых транзакций для быстрого вычисления агрегатных выражений. Для каждой версии правила, находящейся в эксплуатации, создаются свои уникальные таблицы для вычисления агрегатных выражений. Также при полной компиляции в хранилище данных для версии правила обновляется статус на «Loading», который говорит о том, что правило находится в процессе запуска в эксплуатацию. После этого API сервис для работы с рисковыми правилами возвращает ответ веб-приложению. Если возникли ошибки, то веб-приложению отобразит ее причину в специальном всплывающем окне, иначе статус версии правила будет обновлен на «Loading». Внесение изменений запрещено при статусах, отличных от «InDesign». Если другая версия рискового правила уже находится в эксплуатации, то пользователю будет выведено модальное окно с предупреждением, и при его согласии старая версия будет выведена из эксплуатации, а текущая будет запущена в эксплуатацию.

Обновление данных версии рискового правила

Для получения актуальных данных о версии правилах и их автоматического сохранения был реализован таймер, который каждые 10 секунд получает актуальные данные о версии правила из удаленного хранилища данных и сохраняет изменения, сделанные пользователем, при их наличии. На рисунке 19 представлена диаграмма активности UML, отображающая алгоритм срабатывания таймера.

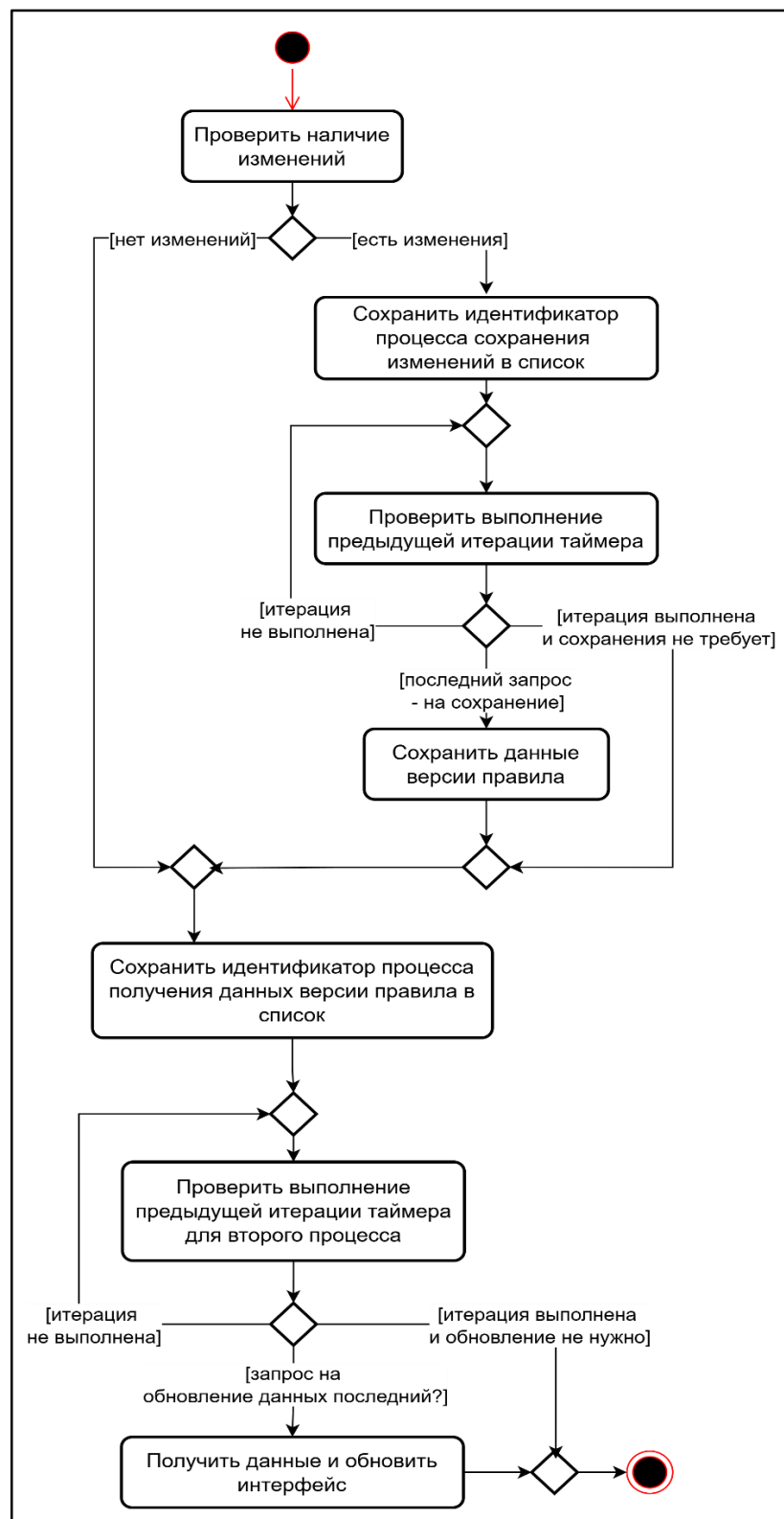


Рисунок 19 – Алгоритм срабатывания таймера

Если пользователь внес изменения, то создается уникальный идентификатор для процесса сохранения данных и добавляется в специальный список. Если предыдущее срабатывание таймера не успело выполниться, то про-

исходит асинхронное ожидание его завершения. В этот момент может возникнуть ситуация накопления нескольких процессов сохранения данных, поэтому выполняется только последний процесс, так как в нем находятся последние изменения пользователя. Аналогично реализовано для процесса для получения данных. Если данные отличаются от текущих, то происходит обновление пользовательского интерфейса. Данная синхронизация исключает возникновение гонки между различными срабатываниями таймера. Также аналогичная синхронизация реализована между запуском версии правила в эксплуатацию, ее выводе из эксплуатации и таймером.

В системе противодействия банковским мошенничествам запуском версий рискованных правил в эксплуатацию занимается отдельный сервис, поэтому данный таймер служит для получения информации, когда версия рискованного правила окончательно перейдет в эксплуатационный режим (статус равен «Production»). Также с использованием этого таймера (получение актуальных данных версии правила каждые 10 секунд) реализован механизм решения конфликтов, когда одновременно несколько пользователей работают над одной версией. Если время изменения, полученное в ответе запроса, больше чем время изменения у пользователя, то появляется специальное окно, которое предлагает пользователю два варианта действий: загрузить изменения с сервера или создать новую версию правила на основе текущей. Если версия правила была удалена, то пользователю предлагается либо воссоздать версию, либо закрыть ее. Если рискованное правило было отправлено в архив, то пользователю предлагается либо восстановить рискованное правило из архива, либо закрыть версию. По итогу не возникнут ситуации, когда пользователь будет долго работать с версией правила, а его изменения будут перезаписываться или вовсе не учитываться.

Тестирование работы рискованных правил

Для тестирования работы рискованных правил была реализована секция «Песочниц». В контексте веб-приложения «Песочница» – структура данных,

привязанная к версии рискового правила, имеющая свой уникальный идентификатор, имя, описание и список тестовых финансовых транзакций (экземпляров потоков данных). На рисунке 20 отображен пример интерфейса с таблицей всех песочниц версии правила.

g Версия 5 5 дней назад mnovokreshenova

Description

Мастер создания правила

```
1 return AReq.acctNumber == 5;
```

Все песочницы 4 new_test2 changena... + Новая песочница

Поиск песочницы

Протестировать все

№	Имя	Время изменения	Последний тест	Изменено пользователем	Строк	Результат теста	Описание
1	changenam1	25.04.2025 18:41:09	28.03.2025 22:15:54	mdremin	34	Ошибка	
2	new_test2	28.04.2025 21:00:42	28.04.2025 21:00:42	mnovokreshenova	3	Пройден	D
3	2	16.01.2025 15:41:50	16.01.2025 15:41:50	mdremin	34	Ошибка	2(Created from 302)
4	4	16.01.2025 15:41:39	16.01.2025 15:41:39	mdremin	419	Ошибка	

Рисунок 20 – Таблица всех песочниц версии рискового правила

В данной таблице отображены имена песочниц, время их изменения, время последнего тестирования, имена пользователей, сделавших изменения, количество строк (количество тестовых финансовых транзакций), результаты тестирования и строковое описание. В песочницах, где тестирование вернуло ошибку, помечаются красным фоном.

В таблице песочниц реализован следующий функционал: открытие данных песочницы, изменение песочницы (имя и описание), дублирование песочницы, удаление песочницы, копирование песочницы в буфер обмена, что позволяет дублировать одну и ту же песочницы в разные версии рисковых правил. Для тестирования работы версии рискового правила на всех пе-

сочницах была реализована кнопка «Протестировать все», которая отправляет запрос для тестирования строк во всех песочницах. Для перемещения между различными песочницами в версии был реализован механизм вкладок, где каждая вкладка является ссылкой на различные песочницы. Редактирование песочниц доступно только для версий рисковых правил, находящихся в режиме разработки.

В интерфейсе отдельной песочницы реализована еще одна таблица, в которой каждая строка представляет из себя экземпляры потоков данных, со своими заполненными данными. На рисунке 21 представлен пример интерфейса песочницы.

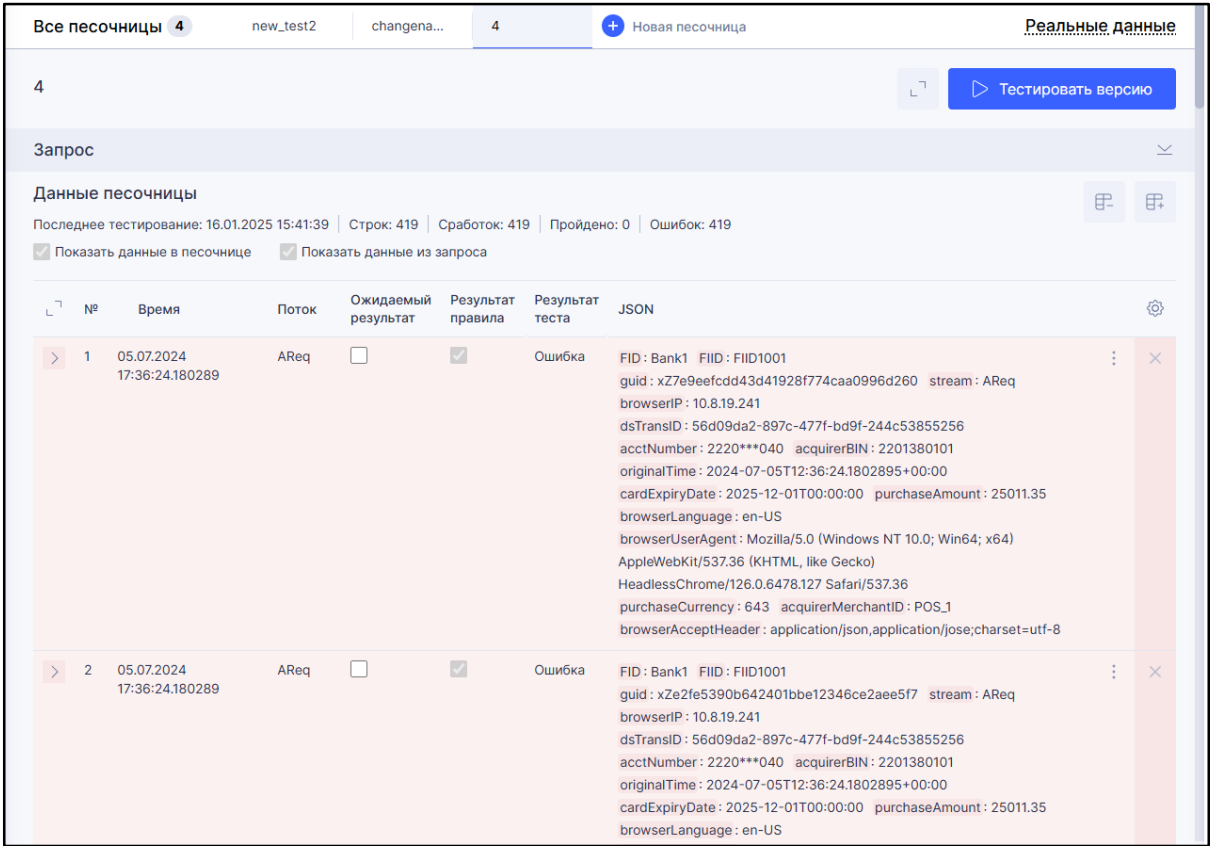


Рисунок 21 – Интерфейс песочницы версии рискового правила

В таблице данных песочницы отображаются время создания транзакции, поток данных (схема финансовой транзакции), ожидаемый результат, результат правила, результат теста и JSON тело, представляющее из себя заполненные поля потока данных. Для тестирования была реализована кнопка «Тестировать песочницу», которая отправляет запрос на API сервис

для проверки работы версии рискового правила. Если рисковое правило определяет финансовую транзакцию, как мошенническую, то для экземпляра потока данных результат теста ошибочен и строка подсвечивается красным фоном, иначе результат теста является пройденным и строка подсвечивается зеленым фоном. Ожидаемый результат – это логическое поле, которое инвертирует результат тестирования.

В таблице реализован следующий функционал: добавление новых строк, удаление строк, дублирование строки, редактирование данных строки, репликация строки, копирование строк в буфер обмена, вставка строк из буфера обмена. Для редактирования полей экземпляра потока данных пользователь может раскрыть необходимую ему строку и обновить нужные поля. Сохранение изменений в тестовых транзакциях песочницы происходит автоматически с помощью таймера, который срабатывает каждые 10 секунд. Редактировать данные песочницы можно только у версий рисковых правил, находящихся в режиме разработки.

Репликация строки – это функционал, который позволяет, используя данные существующего экземпляра потока данных, задать количество повторений, временной шаг, который будет добавляться к времени создания последнего повторения транзакции и правило, по которому определяется, какие поля потока данных будут заполнены значениями в реплицируемых экземплярах потока данных. Например, пользователь может сделать репликацию экземпляра потока данных в песочнице 10 раз, при этом задав интервал времени равный 1 минуте, из-за которого время создания каждого нового повторения будет увеличиваться на 1 минуту по сравнению с предыдущим.

Дополнительно для заполнения песочницы экземплярами потоков данных был реализован функционал, который позволяет формировать запрос для получения совершенных реальных транзакций. На рисунке 22 представлен пример интерфейса для формирования запроса на получение исторических финансовых транзакций и их добавления в песочницу.

Рисунок 22 – Интерфейс запроса для получения исторических транзакций

Для получения необходимых финансовых транзакций пользователь выбирает нужный ему поток данных, диапазон даты и времени, который будет учитывать только те транзакции, которые были совершены в данный интервал времени. Также пользователь может настроить фильтр, который представляет из себя логическую схему, формирующуюся из различных полей выбранного потока данных и операторов. Например, если в работе правила пользователя интересует только те транзакции, где сумма платежа больше 1000 рублей, то этот фильтр поможет ему выбрать только нужные. Кнопка «Выполнить» отправляет запрос на API сервис и в ответе возвращаются финансовые транзакции, которые добавляются в таблицу данных песочницы.

Выводы по третьей главе

В третьей главе были рассмотрены программные средства, которые использовались при реализации веб-приложения. Была описана реализация основных компонентов и функций веб-приложения. Также были разработаны диаграммы UML, отображающие функционал некоторых компонентов веб-приложения и представлены скриншоты пользовательского интерфейса.

4. ТЕСТИРОВАНИЕ

4.1. Функциональное тестирование

Было проведено функциональное тестирование веб-приложения. В ходе данного тестирования проверялось соответствие программного продукта функциональным требованиям. Результаты тестирования приведены в таблице 2.

Таблица 2 – Функциональное тестирование веб-приложения

№	Название теста	Действия	Ожидаемый результат	Тест пройден?
1	Тестирование создания нового рискового правила	Пользователь создает новое рисковое правило с помощью специального модального окна	В боковую панель рисковых правил добавляется новое рисковое правило с нулевой версией	Да
2	Тестирование помещения рискового правила в архив	Пользователь помещает рисковое правило в архив, используя пользовательский интерфейс боковой панели. Разрешено только для правил, которые находятся в режиме разработки	Рисковое правило вместе со всеми версиями помещается в архив. Рисковое правило и все его версии недоступны для редактирования	Да
3	Тестирование восстановления рискового правила из архива	Пользователь восстанавливает рисковое правило из архива, используя пользовательский интерфейс	Рисковое правило вместе со всеми его версиями добавляются в общий список рисковых правил в боковой панели. Рисковое правило и все его версии доступны для редактирования	Да
4	Тестирование удаления рискового правила из архива	Пользователь, находясь в архиве рисковых правил, нажимает кнопку для полного удаления правила	Рисковое правило и все его версии полностью удаляются из системы	Да

№	Название теста	Действия	Ожидаемый результат	Тест пройден?
5	Тестирование назначения метки на рисковое правило	Пользователь переходит в список версий рискового правила и выбирает нужную метку, которую он хочет назначить	Метка добавляется к рисковому правилу и отображается с нужным цветом и описанием возле него	Да
6	Тестирование создания версии рискового правила	Пользователь переходит в список всех версий правила и добавляет новую версию рискового правила на основе актуальной с помощью специального модального окна	В общий список версий рискового правила добавляется новая версия. Номер версии определяется автоматически	Да
7	Тестирование импорта версии рискового правила из файла формата «JSON»	Пользователь с помощью специального окна загружает файл с версией рискового правила и нажимает кнопку «Импортировать»	Если файл являлся валидным, то если рисковое правило с таким именем уже существует, то в список его версий добавляется новая, иначе создается новое рисковое правило с нулевой версией	Да
8	Тестирование экспорта версии рискового правила в файл формата «JSON»	Пользователь нажимает на кнопку «Экспортировать» у версии правила в общем списке версий	Пользователю через браузер загружается искомый файл	Да
9	Тестирование удаления версии рискового правила	Пользователь выбирает пункт «Удалить» у версии правила в общем списке версий	Если версия правила находится в режиме разработки, то она полностью удаляется из системы, иначе операция блокируется	Да
10	Тестирование работы редактора меток	Пользователь редактирует метки в специальном окне «Редактор меток»	Данные меток обновляются в системе	Да

№	Название теста	Действия	Ожидаемый результат	Тест пройден?
11	Тестирование открытия версии рискового правила	Пользователь нажимает на нужную версию в общем списке версий правила	В общий список вкладок веб-приложения добавляется новая с именем рискового правила и его версией. Также под вкладками отображается пользовательский интерфейс для работы с версией рискового правила	Да
12	Тестирование работы редактора кода	Пользователь редактирует исходный код версии правила с помощью редактора кода	Редактор кода корректно подсвечивает лексемы, отображает подсказки и ошибки	Да
13	Тестирование редактирования свойств версии правила	Пользователь изменяет нужные свойства с помощью правой боковой панели «Свойства»	Свойства версии рискового правила корректно обновляются	Да
14	Тестирование сохранения изменений данных версии рискового правила	Пользователь внес изменения в версию рискового правила	Веб-приложения с помощью срабатывания таймера автоматически сохраняет изменения, внесенные пользователем	Да
15	Тестирование обновления данных версии правила в пользовательском интерфейсе	Пользователь открыл интерфейс для работы с версией рискового правила	Веб-приложение с помощью срабатывания таймера автоматически подгружает с серверной части актуальные данные версии рискового правила и корректно обновляет пользовательский интерфейс	Да
16	Тестирование работы веб-формы «Мастер создания правила»	Пользователь редактирует данные версии правила с помощью «Мастера создания правил» и нажимает кнопку «Обновить»	Данные версии правила корректно обновляются и сохраняются. Также исходный код в редакторе заменяется на код, сгенерированный с помощью «Мастера создания правил»	Да

№	Название теста	Действия	Ожидаемый результат	Тест пройден?
17	Тестирование сравнения двух версий одного рискового правила	Пользователь в общем списке версий рискового правила выбирает две нужные версии и нажимает на кнопку «Сравнить»	Отображается пользовательский интерфейс, в котором отображаются два редактора кода с указанием различий исходного кода, и также две древовидные структуры с указанием различий между свойствами версий	Да
18	Тестирование работы боковой панели потоков данных	Пользователь нажимает на кнопку с названием «Потоки» в левой боковой части	Отображается список всех потоков данных системы, их поля и связи.	Да
19	Тестирование проверки работы версии рискового правила с помощью песочницы	Пользователь открыл нужную песочницу и нажал на кнопку «Тестировать версию»	Работа версии рискового правила была успешно проверена на тестовых финансовых транзакциях, добавленных в песочницу.	Да
20	Тестирование создания песочницы в версии рискового правила	Пользователь с помощью модального окна создал новую песочницу	В версию правила добавилась новая песочница	Да
21	Тестирование формирования запроса для добавления в песочницу совершенных реальных финансовых транзакций	Пользователь открывает интерфейс для формирования запроса. Выбирает диапазон даты и времени, поток данных и настраивает фильтр для транзакций	В таблицу данных песочницы добывались новые тестовые финансовые транзакции (экземпляры потоков данных).	Да
22	Тестирование запуска версии рискового правила в эксплуатацию	Пользователь нажимает на иконку ракеты в правом углу интерфейса для работы с версией рискового правила	Последние изменения версии рискового правила сохраняются и ее статус становится равным «Loading». Далее статус становится «Production». Версия недоступна для редактирования.	Да

Все тесты были успешно пройдены, веб-приложение полностью соответствует функциональным требованиям.

4.2. Тестирование системы на адаптивность и кросс-браузерность

Было проведено тестирование веб-приложения на адаптивность, оно должно корректно отображаться на разрешениях ноутбуков и компьютеров. Тесты проводились с помощью инструмента браузера Google Chrome для изменения разрешения экрана. Результаты тестирования приведены в таблице 3. Все тесты были успешно пройдены.

Таблица 3 – Тестирование на адаптивность

№	Ширина экрана в пикселях	Приложение отображается корректно?
1	Более 1400	Корректно
2	От 1200 до 1400	Корректно
3	От 992 до 1200	Корректно

Веб-приложение корректно отображается на всех представленных разрешениях.

Также было проведено тестирование на корректное отображение веб-приложения на различных браузерах. Результаты тестирования приведены в таблице 4. Все тесты были успешно пройдены.

Таблица 4 – Тестирование на кросс-браузерность

№	Браузер	Версия	Приложение отображается корректно?
1	Google Chrome	135.0.7049.115	Корректно
2	Mozilla Firefox	138.0.1	Корректно
3	Microsoft Edge	112.0.1722.64	Корректно
4	Яндекс Браузер	25.2.7.95	Корректно
5	Safari	16.5.2	Корректно

Выводы по четвертой главе

В четвертой главе было проведено тестирования интерфейса веб-приложения на адаптивность и кросс-браузерность. Также было проведено функциональное тестирование веб-приложения, в котором проверялось соответствие разработанного приложения функциональным требованиям. Все тесты были успешно пройдены.

ЗАКЛЮЧЕНИЕ

В рамках данной работы было реализовано веб-приложение «Конструктор правил для автоматического выявления подозрительных транзакций в системе противодействия банковским мошенничествам».

Основные задачи, которые удалось решить.

1. Проведен анализ предметной области. Были проанализированы основные подходы к формированию и применению рисковых правил. Также был проанализирован функционал аналогичных систем.

2. Были определены функциональные и нефункциональные требования к веб-приложению. В рамках этого этапа также была разработана диаграмма вариантов использования UML, которая наглядно представила основные процессы и взаимодействия пользователей с системой.

3. Была спроектирована архитектура веб-приложения, разработаны соответствующие диаграммы UML.

4. Реализовано веб-приложение.

5. Проведено тестирование веб-приложения.

6. Веб-приложение было внедрено в промышленную эксплуатацию в компании ООО «Компас Плюс».

В результате выполнения выпускной квалификационной работы были глубоко изучены фреймворки ASP.NET Core и Blazor WebAssembly, на основе которых было создано веб-приложение. Также был составлен акт о внедрении, который подтверждает успешное использование системы в реальных условиях.

ЛИТЕРАТУРА

1. Батюкова Л. Е., Карлова Т. В. Методика обеспечения безопасности транзакций на основе использования антифрод-системы // Автоматизация и моделирование в проектировании и управлении, 2024. № 1(23). С. 58–64.
2. Бурнашева В. М., Чуручанов И. В. Технологии разработки одностраничного веб-приложения. // Современные научные взгляды в эпоху глобальных трансформаций: проблемы, новые векторы развития, 2021. С. 107–109.
3. Документация по C# | Microsoft. [Электронный ресурс] URL: <https://learn.microsoft.com/ru-ru/dotnet/csharp/> (дата обращения: 10.02.2025 г.).
4. Еремин Д. О., Холмогорова Е. И. Особенности применения Blazor WebAssembly для разработки веб-приложений. // Инновационные технологии в технике и образовании, 2021. С. 320–325.
5. Кряжева Е. В., Васина Т. А. Общие подходы к проектированию веб-приложений. // Заметки ученого, 2021. № 9–2. С. 32–36.
6. Лapidус А. Р. Обзор новых антифрод систем // Аллея науки, 2021. Т. 1, № 2. С. 671–674.
7. Охалов С. К. Современные фреймворки для front end разработки. // Научно-практические исследования, 2020. № 8–1. С. 25–27.
8. Шабан Д. WEBASSEMBLY как успешный пример реализации концепции кроссплатформенности. // Научно-технические инновации и веб-технологии, 2022. № 1. С. 32–38.
9. ASP.NET Core Blazor | Microsoft. [Электронный ресурс] URL: <https://learn.microsoft.com/en-us/aspnet/core/blazor/?view=aspnetcore-7.0> (дата обращения: 10.02.2025 г.).
10. Documentation | Keycloak. [Электронный ресурс] URL: <https://keycloak.org/documentation> (дата обращения: 27.03.2025 г.).

11. Docker documentation | Docker. [Электронный ресурс] URL: <https://docs.docker.com/guides/> (дата обращения: 27.03.2025 г.).
12. FICO Falcon Fraud Manager | FICO. [Электронный ресурс] URL: <https://www.fico.com/en/products/fico-falcon-fraud-manager> (дата обращения: 13.03.2025 г.).
13. Gumus C., Cekerekli S. Fraud prevention and detection systems running with digital onboarding. // Academic studies in engineering, 2024. 15–29 pp.
14. Himschoot P. Microsoft Blazor. // Berkeley, CA: Apress, 2022. 90 p.
15. Monaco Editor API | Microfost. [Электронный ресурс] URL: <https://microsoft.github.io/monaco-editor/docs.html> (дата обращения: 27.03.2025 г.).
16. Smart Fraud Detection | Fuzzy Logic Labs. [Электронный ресурс] URL: <https://fzlabs.ru/applications> (дата обращения: 13.03.2025 г.).
17. Yan Y., Tu T., Zhao L., Zhou Y., Wang W. Understanding the performance of webassembly applications. // Proceedings of the 21st ACM Internet Measurement Conference, 2021. 533–549 pp.